

PARTE A. PROGRAMMAZIONE IN C

SEZIONE 0. INTRODUZIONE AL CORSO

Introduzione alla Programmazione Avanzata e Parallela

X

Introduzione al corso. Motivazioni: l'architettura dei calcolatori influenza la performance. Richiami all'architettura di una CPU. Esempi motivanti: somma di n elementi in un vettore con "step size" diversi; somme su vettori ordinati e non; prodotto tra matrici.

X

0. Voci correlate

- Architettura di Von-Neumann
- Definizione di Numeri Macchina

1. Introduzione al corso

Nel corso di *programmazione avanzata e parallela* vedremo due aspetti della programmazione:

- Come si può gestire progetti complessi e capire il paradigma OOP e della programmazione funzionale. Molto utile in certe applicazioni, come *autograd* ("automatic differentiation") ("*Avanzata*")
- Sfruttare il fatto che ogni dispositivo possiede più *core*, ovvero parallelizzare i processi ("*Parallela*"). Notiamo che parallelizziamo *solo* a livello di *singola macchina*

Nella prima parte del corso vedremo la parte "*parallela*" del corso, dopodiché vedremo la parte "*avanzata*". Nella prima parte si vedrà il linguaggio *C*, dopodiché nella seconda parte si andrà a vedere prevalentemente il linguaggio *Python*. In particolare, ci sarà una fase *intermedia* in cui si studia come *Python* sia in grado di "*invocare*" il linguaggio *C* (o simili)

X

2. Effetti dell'Architettura sulle Performance

Iniziamo dunque la prima parte sul corso *motivando* il fatto che bisogna studiare e capire il funzionamento dell'architettura degli elaboratori per poter osservare le *performance* dei programmi.

Ancora più precisamente, il principio è come segue:

Principio

L'architettura dei calcolatori influenza le prestazioni dei programmi che scriviamo

Per capire *come mai* vale questo principio, dobbiamo prima richiamare un paio di concetti relativi all'architettura dei calcolatori (in particolare vedendo i *processori*). La CPU (il "*processore*") è composta da molte parti "*essenziali*" per poter eseguire un programma, tra cui:

- Decodifica delle istruzioni (*Decode*)
- Calcolo numerico con numeri macchina (*Floating Point*)
- Operazioni aritmetiche (*ALU*)
- Comunicazione con la memoria (*Load/Store*)

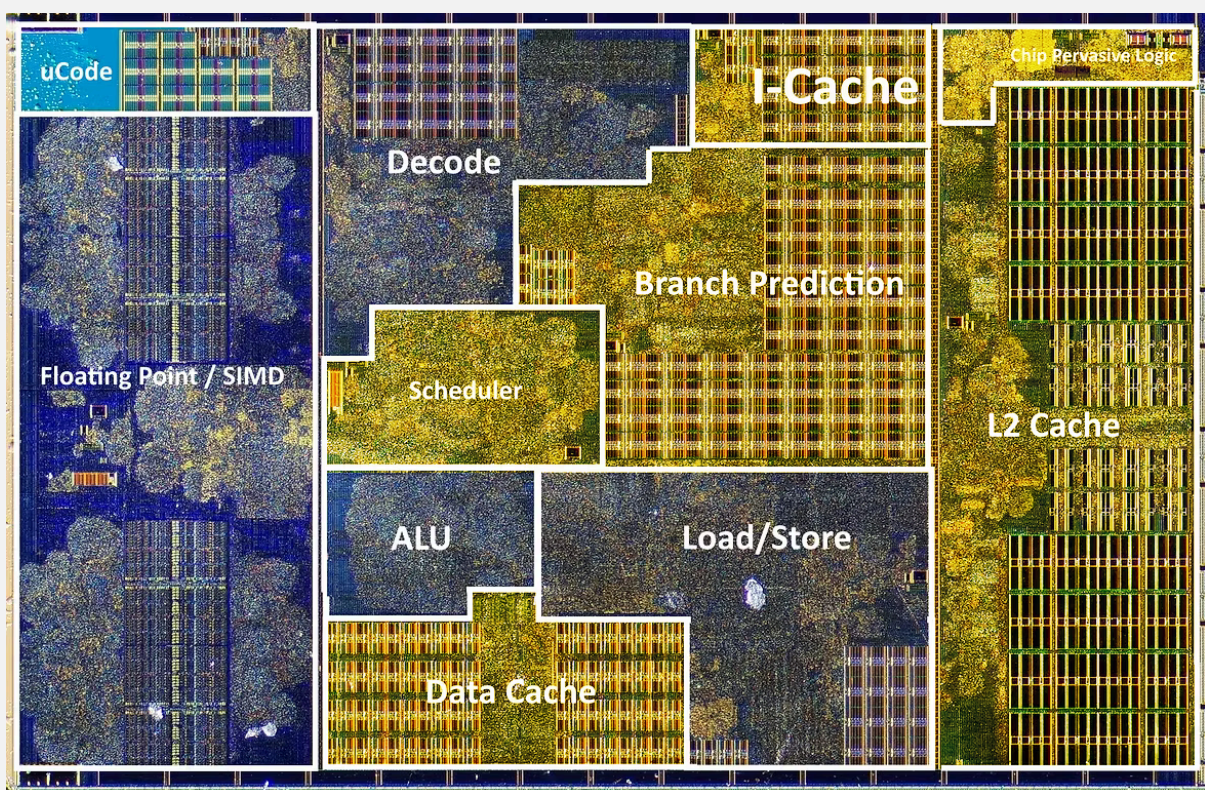


Figura: Diagramma del core di Zen 2 CPU

Tuttavia, guardando un processore reale notiamo che ci sono moltissime altre parti "non essenziali" (nella figura sopra, le parti in *giallo*), ma utilissimi per la prestazione dei programmi. Negli esempi successivi vedremo degli esempi in cui vengono utilizzati queste parti.

#Esempio

Esempio 1 (Calcolo di n vettori con passi incrementali diversi).

Sia A un vettore (con valori "random" infiniti). Supponiamo di voler calcolare la somma delle sue n entrate, ognuna distanziata di 2^k blocchi. I tempi di calcolo sono diversi o

uguali per k ?

Risposta: Effettuando una semplice simulazione (vedere [Lecture 01/mem_latency.c](#) nei file del corso), si scopre che ci sono delle differenze significative; infatti, da $k = 0$ a $k = 17$ si osserva uno scarto nei tempi dell'esecuzione (in secondi) di ordine 10^3 (più precisamente, da circa 10^{-2} a 10^1).

#Osservazione

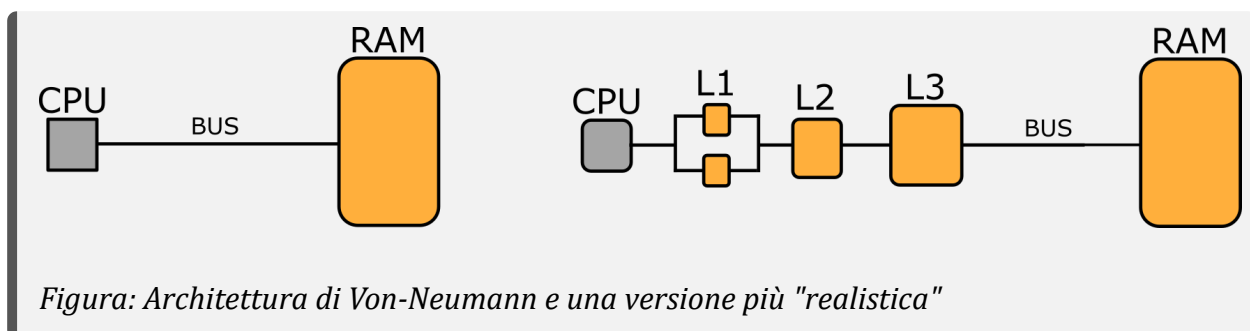
Osservazione 2 (Cache dell'architettura).

Come mai è verificata una roba del genere? Torniamo a vedere l'architettura dei calcolatori, in particolare l'architettura di *Von-Neumann*.

Il modello di Von-Neumann prevedeva che la *CPU* fosse collegata all'unità di memoria (la *RAM*) mediante il *bus*. Tuttavia, ciò non è sufficiente per capire il fenomeno dell'esempio precedente.

Il problema di questo modello consiste infatti nel fatto che la *CPU* attende l'accesso alla memoria per poter eseguire istruzioni, che ha una latenza "molto maggiore" di eseguire istruzioni. Pertanto ci sono altre unità di memoria chiamate "cache" (denominate con L1, L2 e L3 cache; occasionalmente anche L4) che hanno una latenza molto minore di quello della RAM (tuttavia hanno meno spazio!).

In particolare, i dati vengono trasferiti nelle cache "a blocchi", quindi sono favoriti gli accessi più vicini. Ciò spiega quindi il fenomeno appena osservato.



#Esempio

Esempio 3 (Somma di due vettori).

Supponiamo di avere due *vettori* con dimensione fissa e di voler calcolare le loro relative somme degli elementi superiori (o inferiori) una certa soglia. Supponiamo, inoltre, che uno sia ordinato e l'altro non ordinato.

Ci sono differenze nei tempi di esecuzione?

Risposta: Anche qui i tempi sono diversi, con i vettori più ordinati che godono delle prestazioni migliori (vedere [Lecture 01/branch_pred.c](#))

#Osservazione

Osservazione 4 (Branch prediction).

Le CPU moderne hanno più istruzioni in esecuzione contemporaneamente; in particolare, sotto certe condizioni posso "anticipare" l'esecuzione di alcune istruzioni.

Tuttavia, se ho dei *salti condizionali* (come dei blocchi `if-else`), bisogna effettuare una *previsione* in cui si sceglie una *branch* da eseguire. Questa previsione ("*guess*") si basa tipicamente sull'osservazione dello storico (vedremo com'è realizzato tale processo). In particolare, se la previsione è corretta, allora ho risparmiato tempo; tuttavia, se è errata allora non si considererà più la parte eseguita e risulterà di aver perso tempo e risorse computazionali.

Questa tecnica è particolarmente utile quando la previsione è molto semplice, come appunto nell'esempio di prima con i vettori ordinati.

#Esempio

Esempio 5 (Moltiplicazione delle matrici).

Siano $A \in \mathbb{R}^{n \times p}$ e $B \in \mathbb{R}^{p \times m}$ e supponiamo di voler calcolare il prodotto riga-per-colonne AB . C'è una differenza di prestazioni nel calcolo del prodotto a "*parità di algoritmo*" in linguaggi di programmazione diversi?

Risposta: Sì, nonostante il fatto che l'algoritmo "classico" sia in $O(n^3)$, ci sono delle costanti che dipendono dall'implementazione dell'algoritmo. In particolare, si osserva che in Python l'algoritmo ci mette 8 secondi, invece in C solo 0.18 secondo (circa) (vedere [Lecture 01/matmul.c](#) e [Lecture 01/matmul.py](#))

SEZIONE A. GESTIONE DEI PROGETTI IN C

Extern e Header

X

Primo step dell'organizzazione dei progetti: gestire strutture e funzioni condivise.

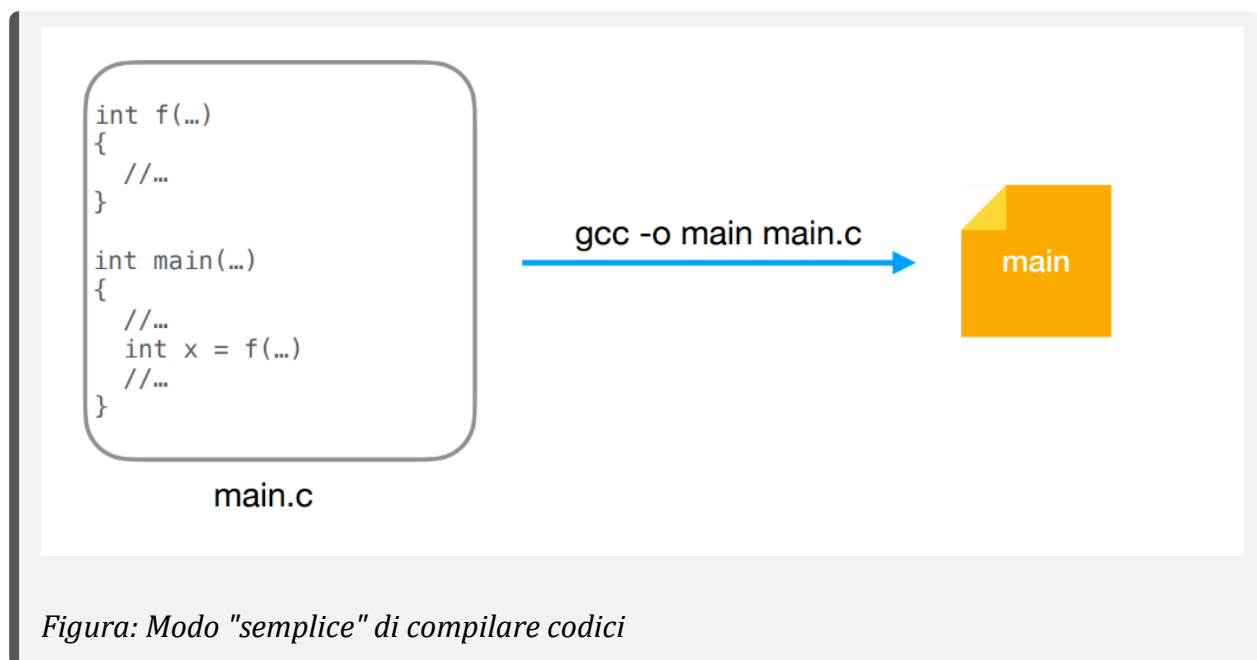
X

0. Voci correlate

- Introduzione alla Programmazione Avanzata e Parallela
- Architettura Base del Calcolatore

1. Compilazione di Programmi

RICHIAMO. Ricordiamo che fino ad'ora siamo in grado di compilare codici scritti in un *linguaggio sorgente* (come C, Fortran, eccetera) in *linguaggio macchina* mediante il compilatore.



Tuttavia, siamo solo in grado di compilare *singoli file sorgenti*. Nella realtà, la situazione è ancora più complicata! Infatti, ci sono situazioni in cui:

- Voglio organizzare un progetto in più file, per un discorso di "modularità"
- Condivido quindi delle strutture dati e delle funzioni in comune
- Pertanto sarebbe opportuno definire funzioni o definizioni in file separati...

Alcuni esempi già implicitamente noti sono `printf`, `malloc`, eccetera...; questi provengono infatti dalla libreria `stdio` ("Standard I/O").

Il nostro obiettivo è quindi di trovare un modo per gestire il progetto nel modo di cui sopra, in particolare voglio una fase "intermedia" in cui il *codice macchina* effettua dei branch verso definizioni esterne, che poi vengono riassemblati nella fase del *linking*.

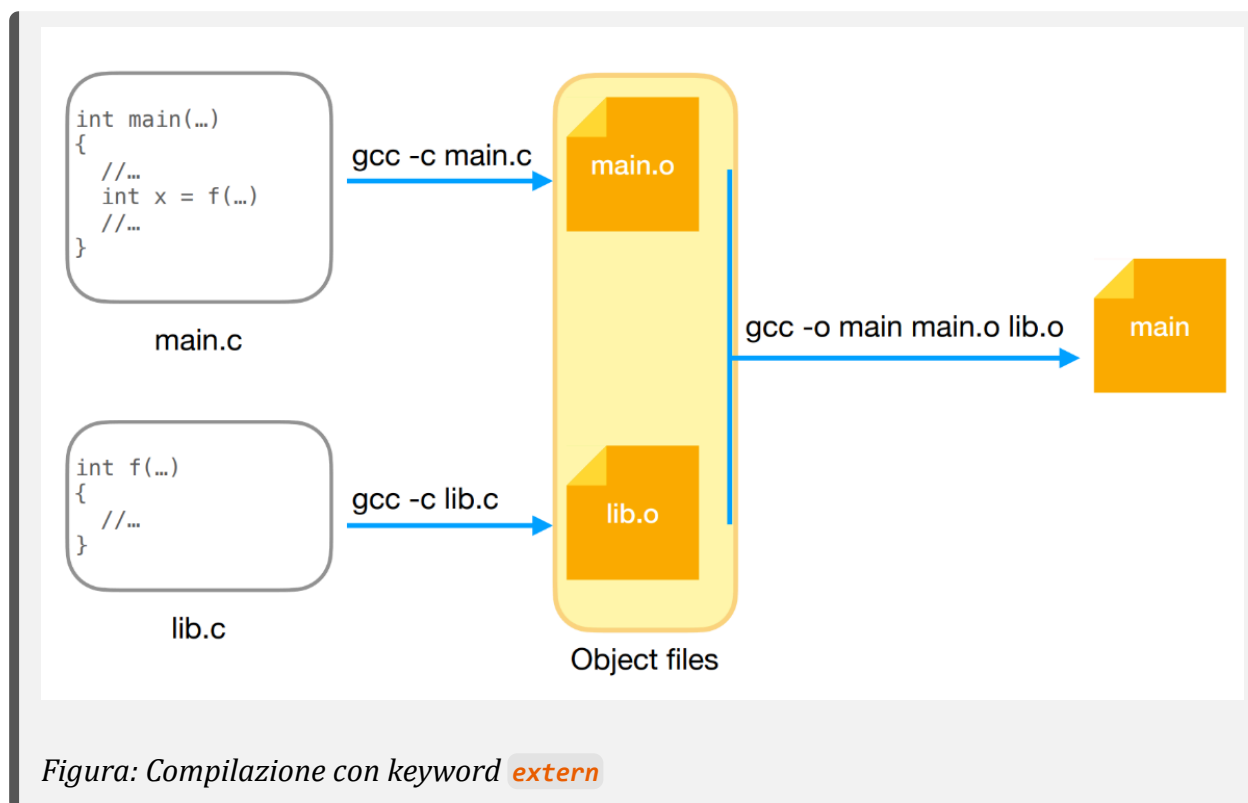
X

2. Extern

Un modo per farlo è con la funzionalità `extern`, per le funzioni.

- L'idea di base è quello di dire che *esiste una certa funzione*, tuttavia non lo implemento nel codice
- La sintassi consiste nel posizionare la keyword `extern` prima della funzione, come ad esempio `extern int f(int x, int y);`
- *N.B.* Funziona SOLO con le funzioni! Quindi non strutture... (vedremo dopo con gli *header*).
- *Osservazione.* La definizione può essere duplicata su più file, a patto che la definizione è consistente

In questo modo, il processo di compilazione viene ulteriormente spezzato in una fase "intermedia" di compilazione, in cui si hanno gli *object files* che contengono dei "riferimenti" alle funzioni esterne.



X

3. Header

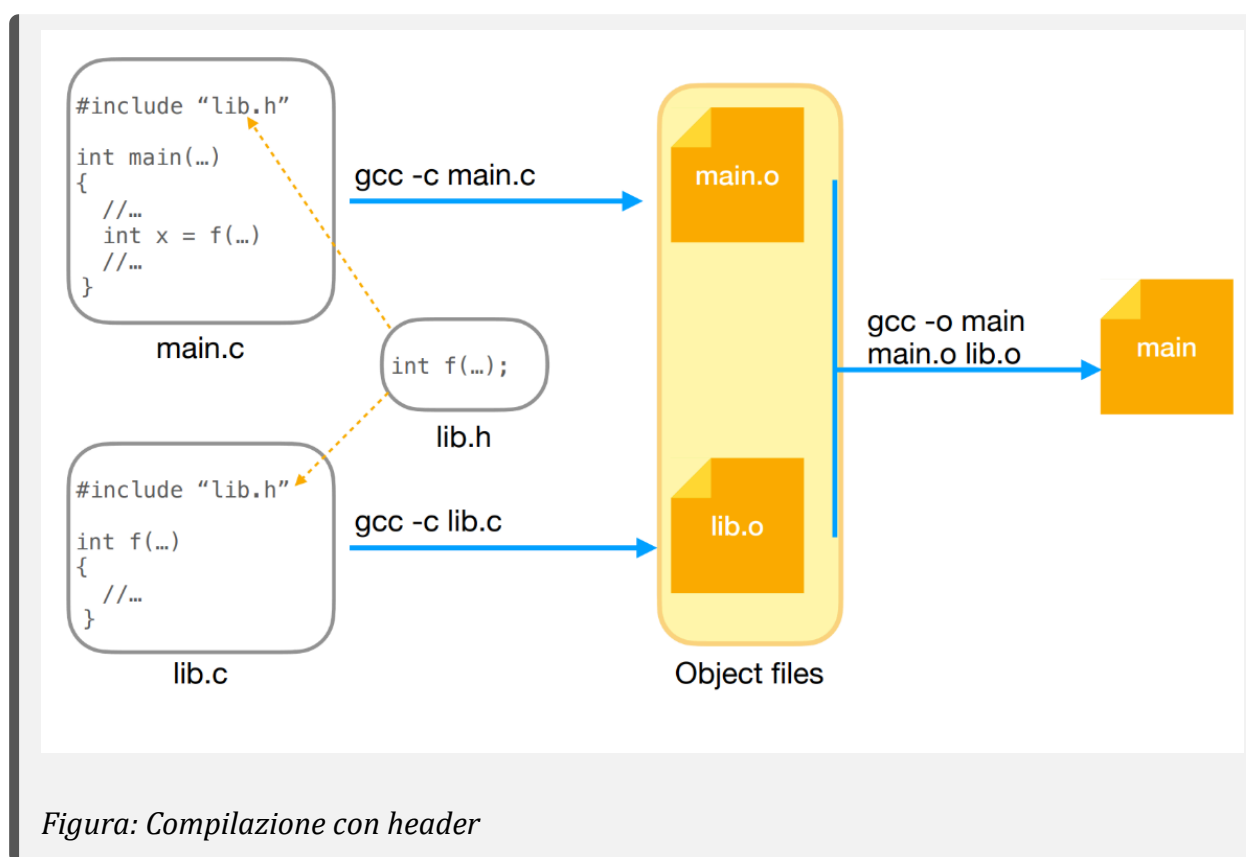
Il fatto che le strutture (struct) vengano definite in *ogni file* non è ottimale, infatti crea problemi di mantenibilità in quanto dei cambiamenti minimi alla definizione delle struct (come l'ordine dei campi) può "rompere" un codice. Per ovviare, dovrei propagare i cambiamenti; tuttavia, immagina di farlo in un progetto con 100 file di codici sorgenti!

Un modo più *comodo* per gestire questo aspetto sono gli *header*, che sono dei file che dicono "*quali sono le strutture e le funzioni utilizzabili*". L'idea di base è quello di contenere la *dichiarazione* (NON l'implementazione! Vengono inclusi in file `.c` a parte per un discorso di "good practice") di funzioni e *definizione* di strutture.

Solitamente sono in formato `.h` e vengono inclusi con le direttive `#include "file.h"` o `#include <file.h>`.

Osservazione. `#include "file.h"` quando la header è contenuta localmente (ovvero sulla stessa cartella del codice sorgente o altrove, da specificare!) e invece `#include <file.h>` quando si vuole usare dei path predefiniti in cui cercare gli header.

Come con la funzionalità `extern`, la compilazione del progetto si spezza in più parti.



In un certo senso, gli header formano un contratto da rispettare dai file sorgenti.

Q. Se includessimo un header più volte?

Per le dichiarazioni di funzioni non c'è problema, siccome rimangono tutte consistenti tra loro; tuttavia il discorso è diverso per le *definizioni di struct*, in quanto vengono duplicate!

Oggi un problema del genere è facilmente rilevabile con delle macro del preprocessore C, quindi teoricamente basterebbe "togliere" la parte in cui si viene fatta l'inclusione multipla.

Alternativamente, per evitare questo problema intervenendo sull'header stesso, si può usare il direttivo `#ifndef ...` che - a parole povere - dice di ignorare un certo blocco del codice se è già stato definito una certa "keyword".

Esempio.

```
#ifndef _LIB_H
#define _LIB_H

int f(...);

#endif
```

C

Librerie C

X

Ulteriore passaggio nella gestione dei progetti in C: creare librerie. Librerie statiche e dinamiche: come si creano, come si utilizzano, vantaggi e svantaggi.

X

0. Voci correlate

- Extern e Header

1. Creazione delle Librerie: Overview

Un passaggio successivo nella *gestione dei progetti in C* è la creazione delle librerie, per renderlo disponibile senza dover necessariamente condividere il codice sorgente.

Generalmente abbiamo due possibilità:

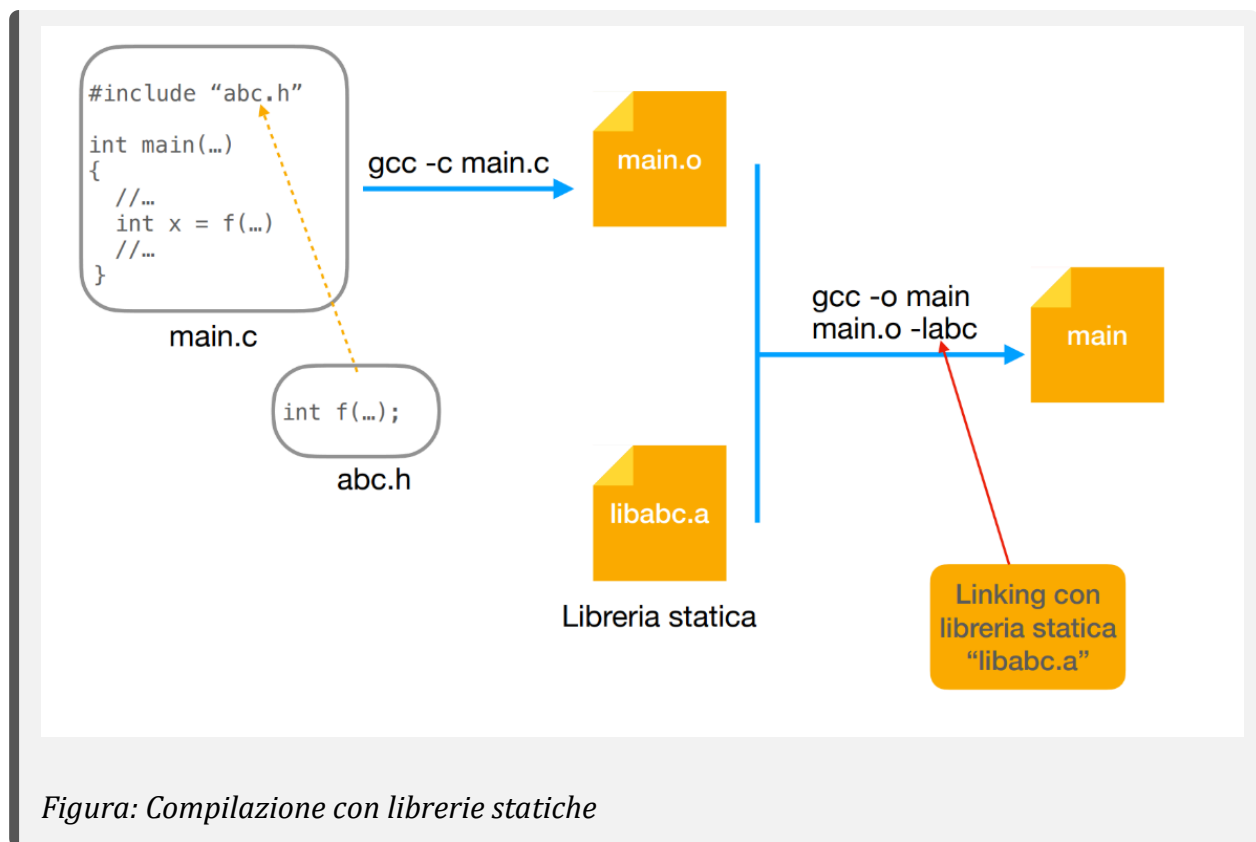
- *Librerie statiche*: Vengono incluse nel programma al momento del *linking*, quindi in fase di compilazione. Viene creato col comando `ar`
- *Librerie dinamiche*: Quando il programma viene eseguito, si cerca e si carica la libreria dinamica. Viene creato col comando `nm`

N.B. I comandi di creazione si riferiscono all'ambiente UNIX dei sistemi operativi

X

2. Librerie Statiche

Come accennato prima, le librerie statiche vengono incluse nella fase di *compilazione*. Dunque la compilazione di un programma si spezza come segue:



Creazione. La creazione di una libreria statica avviene nei seguenti passaggi:

1. Creare gli object file opportuni
2. Assemblare i file assieme in una libreria
3. Creare un "indice" dei simboli della libreria

Quindi un esempio dei comandi che eseguono la procedura è:

1. `gcc -c file1.c file2.c`
2. `ar r lib.a file1.o file2.o`
3. `ranlib lib.a`

Alternativamente si può fare assieme gli ultimi due passaggi con un singolo comando:

1. `gcc -c file1.c file2.c`
2. `ar rs lib.a file1.o file2.o`

Utilizzo. Per utilizzare una libreria statica, bisogna specificare al compilatore sia il *path* in cui cercare la libreria (`-L<path>`) che la libreria con cui effettuare il linking (`-l<library-name>`).

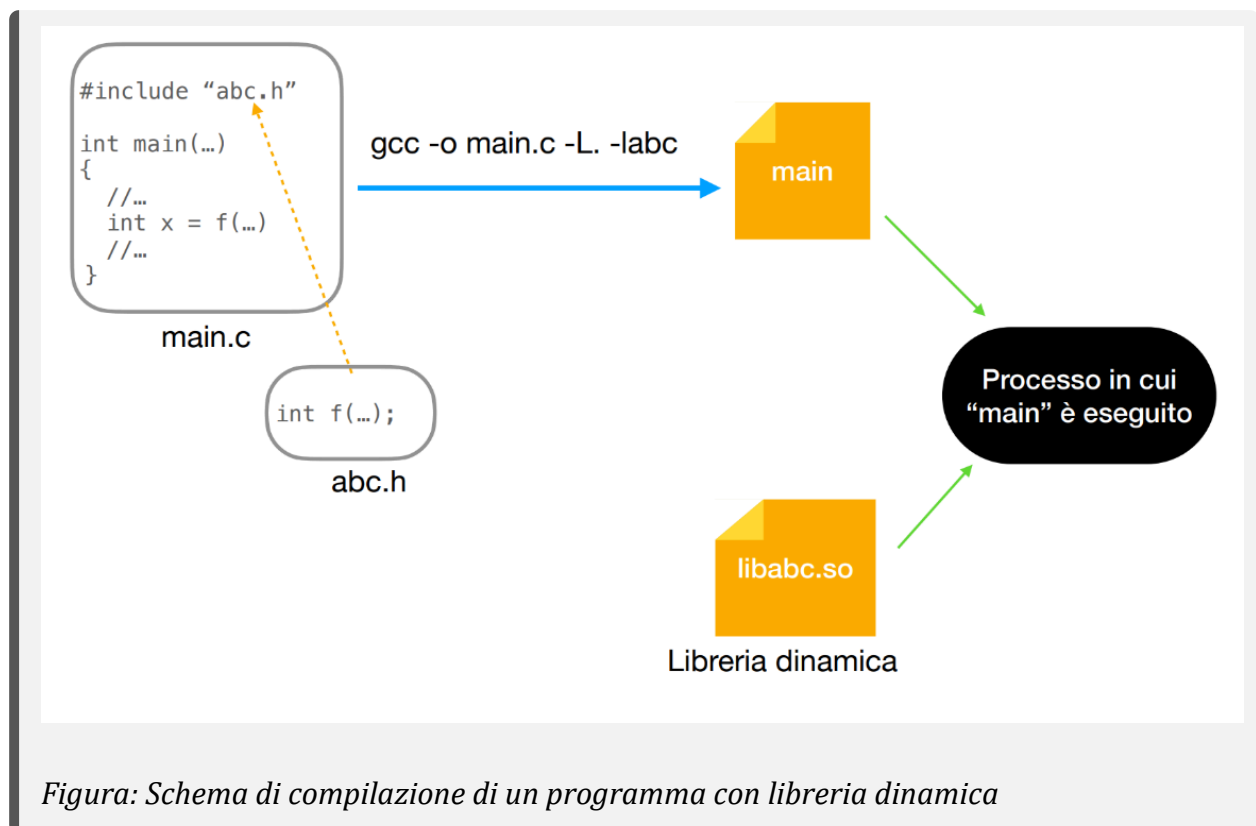
- Esempio: `gcc -L. -o main main.c -labc`

3. Librerie Dinamiche

Motivazioni. Il problema con le librerie statiche consiste nel fatto che posso aggiornare le librerie senza dover ricompilare i programmi da capo. Quindi una libreria dinamica, che vengono caricate *quando il programma viene eseguito*, presenta i seguenti vantaggi:

- Aggiornabili senza la necessità di ricompilare tutto
- Si preservano in un'unica copia (siccome tutti i programmi usano la stessa libreria), risparmiando dunque tempo ed evitando possibili conflitti
- Vengono caricate *solo quando* sono necessarie

Quindi il processo di compilazione rimane invariata, in quanto la libreria dinamica interviene nella fase di *esecuzione* del codice macchina.



La creazione e l'utilizzo delle librerie dinamiche è difficile in quando bisogna tenere conto di due aspetti:

- La "*position independence*", ovvero il codice deve poter eseguire indipendentemente dagli indirizzi di memoria
- Il loader deve sapere dove *trovare* la libreria dinamica; solitamente in path definiti come *variabili d'ambiente*

Creazione. La creazione avviene in tre step:

1. Compilazione "*position independent*" del codice
2. Creazione della libreria
3. Compilazione del programma con la libreria dinamica

Esempio.

1. `gcc -fPIC -c abc.c`
2. `gcc -o libabc.so -shared abc.o`
3. `gcc -o main main.c -L. -labc`

N.B. In ambiente UNIX le librerie dinamiche hanno l'estensione `.so`

Osservazione. Si può vedere quali librerie dinamiche un programma compilato necessita col comando `ldd <executable>`

Utilizzo. In Linux la variabile d'ambiente che contiene le librerie dinamiche è `LD_LIBRARY_PATH`.

- Per vedere cosa contiene basta fare un'echo: `$LD_LIBRARY_PATH`
- Si può aggiungere dei path alle variabili con `export LD_LIBRARY_PATH=<my_directory>:$LD_LIBRARY_PATH`

Nota (facendo gli esercizi). Servono anche le opzioni `-Wl,-rpath,.` (???)

Opzioni Comuni del Compilatore

X

Opzioni comuni del compilatore. Opzione `-On` per ottimizzazioni del programma compilato. Opzioni per debugging del codice. Opzioni per aggiungere warning.

X

0. Voci correlate

- Programmazione Avanzata e Parallela

1. Opzioni di Ottimizzazione

Il compilatore *C* (noto come `gcc`) presenta moltissime opzioni per compilare codici. Faremo una breve carrellata di queste opzioni.

L'opzione `-On` ("O grande, n) con $n = 0, 1, 2, 3, s$ ci permette di specificare in quale "grado" ottimizzare il codice compilato.

- Intuitivamente, per $n = 0$ non ho nessuna ottimizzazione; invece $n = 3$ ho il *massimo* livello di ottimizzazione (solitamente)
- L'opzione $n = s$ ottimizza lo spazio

L'opzione `-march=<name>` indica l'architettura target della compilazione, quindi "decide" su quali macchine può essere eseguito.

- In particolare, `-march=native` per utilizzare *tutte* le estensioni (quindi ottimizzazioni) dell'architettura specifica della macchina

X

2. Debugging

L'opzione `-g` compila il codice coi *simboli di debug*, ovvero mantengo cose come nomi delle variabili; serve per i programmi di debugging

L'opzione `-profile-generate` compila per permettere il *profiling*, ossia per includere dei file aggiuntivi per ottimizzare codici. Può aiutare le ottimizzazioni mediante `-fprofile-use`

Esempio. Un certo path di un flow if-else è quasi considerato, quindi col profiling il branch predictor può effettuare una guess sul branch

X

3. Warning

Alcune opzioni per abilitare le *warning*, per rendere il codice più mantenibile.

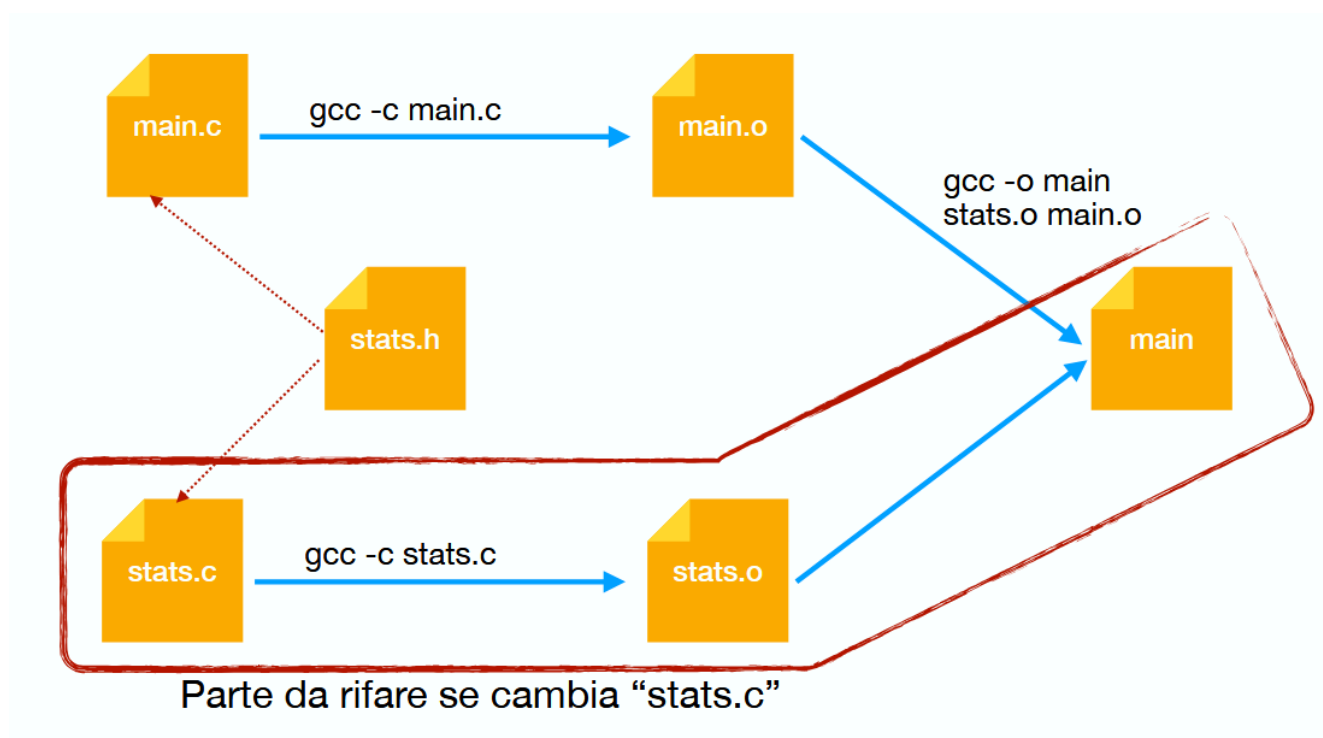
- `-Wall` ("Warning all") abilita i warning *ragionevoli* (NON TUTTI!)
 - Purtroppo non significa che vi mette un muro nel sorgente...
- `-pedantic` per abilitare tutti i warning *relativi a dei standard* (quindi NON tutti!)
- `-std=<...>` per specificare lo standard relativo ai warning "*pedantici*"
- `-Wextra` ancora più warning di `-Wall` (comunque non tutti!)

0. Voci correlate

- Introduzione alla Programmazione Avanzata e Parallela

1. Motivazioni per Build Automation

Supponiamo di gestire un progetto con *tantissimi* (ma tantissimi) file e di aver già compilato tutto. Se modificassimo *un solo file*, cosa dovrei fare? Ovviamente, si vorrebbe solamente ricompilare i file che sono affetti da questa modifica, ovvero il *necessario* indispensabile.



Quindi un sistema di *build automation* automatizzerebbe il processo di compilazione, controllando "autonomamente" quali sono le parti da compilare o meno.

Ci sono moltissimi software di *build automation*, ognuno specifico ad un linguaggio di programmazione. Alcuni esempi sono:

- *CMAKE* per generare *script* e fare build con altri tool
- *Autotools* per generare i "makefile" (vedremo)
- *Bazel* per Java o qualcosa che c'entra con Google (generalmente)
- *Gradle* per Java

- *Make* per C

Noi approfondiremo molto bene il sistema *make*.

Make e Makefile

X

Tutto su make e makefile. Sintassi di un makefile: target, requisiti e comandi. Usare makefile per effettuare la "pulizia" di un programma. Variabili di un makefile. Regole template. Wildcards. Regole condizionali.

X

0. Voci correlate

- Sistemi di Build Automation

1. Introduzione a Make

Make è uno strumento di *build automation* che permette di specificare tre aspetti di compilazione di un file:

- *Obbiettivo ("target")*, ovvero *cosa* vogliamo compilare
- *Prerequisiti*, ovvero *cosa serve* per compilare il target
- *Comandi*, ovvero quali sono i comandi da eseguire per compilare il target

Ogni tripla forma una "*regola*", che va a formare un *obbiettivo* secondo dei *prerequisiti* eseguendo alcuni *comandi*.

La prima versione di Make risale al 1976 (quindi oggi ha esattamente 50 anni!).

L'idea di base è quello di creare un *makefile* che contiene le tre cose di cui sopra (target, prerequisiti e comandi) e invocandoci *make* si può costruire la versione aggiornata del programma.

X

2. Makefile

Sintassi. Come detto prima, un *makefile* è composto da *obbiettivo*, *prerequisiti* e *comandi*. La sintassi è come segue:

```
target: req1 req2 ...  
    command1  
    ...  
    commandn
```

Notiamo che prima del comando devo obbligatoriamente *indentare* col TAB. Assicurarsi la correttezza della sintassi è *molto importante*!

Regole. Nell'esecuzione di *make* su un *makefile* abbiamo le seguenti regole:

- Se modifico un prerequisito di un target, devo ricostruire il target
- Verifico i prerequisiti *ricorsivamente*, ossia:
 - Se non esiste, cerco una regola per crearlo
 - Se esiste, verifico se devo ricostruirlo o meno (verificando appunto i suoi prerequisiti *ricorsivamente*)

Per invocare un *makefile*, si usa il comando `make`. In particolare:

- `make <target>` per costruire un target specifico (come *main*)
- `make -f <makefile_name> <target>` per specificare *quale* makefile usare
- `make` per costruire il target *"all"*

Osservazione. Un target comune è *"clean"*, che non corrisponde a nessun file ma esegue una serie di comandi per *pulire i file intermedi*. Un esempio è associarci il comando `rm *.o`.

Osservazione. Se voglio avere un target "fittizio" ma c'è già un file che si chiama col nome del target "fittizio", allora bisogna usare prima la direttiva `.PHONY` per considerare il target *"mai aggiornato"*

Osservazione. Se si inserisce il simbolo `@` prima di un comando, invocando il comando `make` non viene stampato il comando eseguito a schermo

X

3. Variabili Makefile

Q. Notiamo che nel *makefile* possiamo anche generare librerie. Come possiamo specificare in una maniera *"modulare"* la directory? Analogamente, come posso configurare in una maniera "globale" le opzioni di compilazione?

Ovviamente fare un *copia-e-incolla* "ovunque" è una pessima idea, in quanto rende il makefile pesante da mantenere.

Una soluzione "facile" è data dalle *variabili del makefile*.

Sintassi. Le variabili vanno *dichiarate* e poi *usate*. Quindi:

- `NAME=VALUE` per dichiarare. Alternativamente, dalla linea di comando si può specificare direttamente la variabile scrivendo `make NAME=VALUE ...` (molto utile per debugging!)
- `${NAME}` per utilizzare

Esempio. Un esempio molto comune è quello di specificare i *flag di compilazione*. Quindi ad esempio ho `CFLAGS=-O3` e poi successivamente da qualche parte scrivo come comando `gcc ${CFLAGS} -o file file.o`

Osservazione. Possiamo utilizzare le variabili *ovunque*! Quindi non solo nel comando da eseguire, ma anche nei *prerequisiti* e nel *target*, basta usarli in una maniera consistente.

Variabili Automatiche. Nel makefile esistono sempre *alcune variabili*, tra cui:

- `$@` denota il *target*
- `$^` denota la *lista dei prerequisiti*
- `$<` denota il *primo prerequisito* relativo al target

Questi sono utili, in quanto ci permette di evitare di scrivere dei nomi già presenti nei target e nei prerequisiti. In questo modo, li metto una sola volta e me li tengo ovunque...

Esempio. Per convertire un file C in un file object si può usare la seguente regola:

```
file.o file.c
gcc ${CFLAGS} -c $<
```

X

4. Regole Template

Osservazione. L'esempio di prima può essere generalizzato per il nome del file, ovvero notiamo che possiamo evitare di ripetere la parte `file`, che sta prima dell'estensione.

Quindi le *regole template* sono delle regole "simili tra loro" in cui si cambia solo il nome. Per farlo possiamo usare `%` per indicare una "*qualsiasi sequenza di caratteri*" con cui fare il matching.

Esempio. Esempio di prima generalizzato:

```
%.o %.c
gcc ${CFLAGS} -c $<
```

Con le *regole template* abbiamo inoltre una variabile aggiuntiva: `$(*)` serve per denotare il "*matching*" effettuato con `%` (ovvero la parte ripetuta).

Osservazione. Ricordiamo che Makefile non è limitato alla costruzione dei progetti in C; è un sistema di *build automation generico*, funziona anche per compilare documenti in *Pandoc*!

X

5. Wildcards

Le wildcards sono delle "variabili" che rappresentano una *lista di file che rispettano un certo pattern*. La sintassi è simile a quello delle variabili, solo che si usano e per usarli bisogna

inserire il termine `wildcard` all'inizio.

Ad esempio `${wildcard *.c}` rappresenta tutti i file C nella cartella presente.

Per approfondire su come scrivere i pattern, vedere il sito ufficiale della GNU:

https://www.gnu.org/software/make/manual/html_node/Functions.html

X

6. Regole Condizionali

Idea. In certi casi i comandi possono essere diversi a seconda di alcuni parametri, come ad esempio il compilatore scelto.

Abbiamo due regole condizionali:

If-else: Come la conosciamo. La sua sintassi è

```
ifeq "string1" "string2"
...
else
...
endif
```

Ifdef: Eseguire un blocco se una variabile è definita o meno. Molto utile per debugging! (per verificare se ho inserito un flag di debugging o meno)

```
ifdef "var"
...
else
...
endif
```

X

7. Un paio di osservazioni sul makefile

Osservazione. Ogni riga di comando scritto è *indipendente* dall'altra, quindi non vengono eseguiti assieme. Se vogliamo eseguire alcuni comandi "in sequenza", allora bisogna separarli col punto e virgola `;`.

Osservazione. Importantissimo indentare i comandi! Niente spazi!!!

Osservazione. Il flag `-s` di `make` serve per eseguire la costruzione senza eseguire i comandi veramente (utile per capire se funziona o meno il makefile)

SEZIONE B. PRESTAZIONE DEI PROGRAMMI E DEL PROCESSORE

Pipeline

X

Metriche di performance: latenza e throughput. Richiami al ciclo fetch-decode-execute: calcolo del throughput. Tecnica del pipeline. Tecniche per migliorare il throughput: esecuzione superscalare, esecuzione out-of-order. Problema delle "pipeline bubbles": definizione di pipeline hazard, tre tipologie (structured, data e control).

X

0. Voci correlate

- Introduzione alla Programmazione Avanzata e Parallela
- Architettura di Von-Neumann

1. Misure di Prestazione

Latenza

La *latenza* è il tempo che intercorre da quando un'operazione inizia e quando viene completata. Può essere misurata in *termini di tempo* (nanosecondi) ma tipicamente viene misurata in *numero di cicli (CPU clock)*

Throughput

Il *throughput* è il numero di operazioni completate in una quantità di tempo. Tipicamente misurato in termini di *IPC* (Instructions per Cycle), ossia il numero di istruzioni completate ad ogni ciclo di clock.

Ricordiamo che per eseguire un'istruzione, la CPU esegue il seguente "*ciclo base*":

1. *Fetch*: caricare l'istruzione dalla memoria
2. *Execute*: decodificare l'istruzione
3. *Execute*: eseguire l'istruzione
4. *Memory*: accedere alla memoria (eventualmente)
5. *Write*: scrivere i risultati nei registri

Esempio. Supponendo che la CPU esegua le istruzioni uno ad uno, abbiamo che il throughput è di 0.2 istruzioni per ciclo.

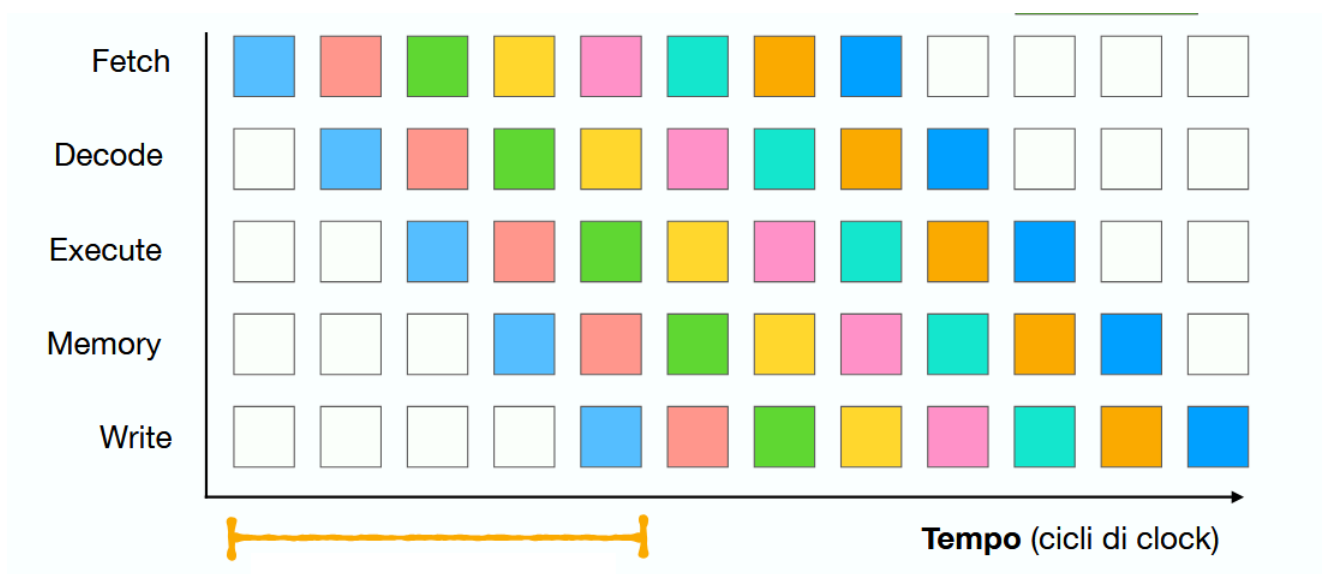


X

2. Pipeline

Osservazione. Nel primo calcolo del throughput, notiamo che alcune delle fasi vengono compiute da parti di CPU diverse; quindi solitamente sono *libere*! Si potrebbe fare di meglio eseguendo alcuni parti "*in anticipo*", se conosciamo qual è la prossima istruzione da eseguire.

Pipeline. Infatti, si potrebbe fare proprio questo e si otterrebbe (circa) un throughput di 1.0, ovvero eseguiamo un'istruzione per ogni ciclo di clock. Questa è proprio la tecnica del *pipeline*.



Osservazione. Miglioriamo solo il throughput, non la latenza!

Setup Cost

Definiamo il *setup cost* come il tempo necessario per far "*riempire*" la pipeline, raggiungendo una quantità stabile di throughput.

Ovvero il setup cost è, nel nostro caso, di 5 cicli di clock.

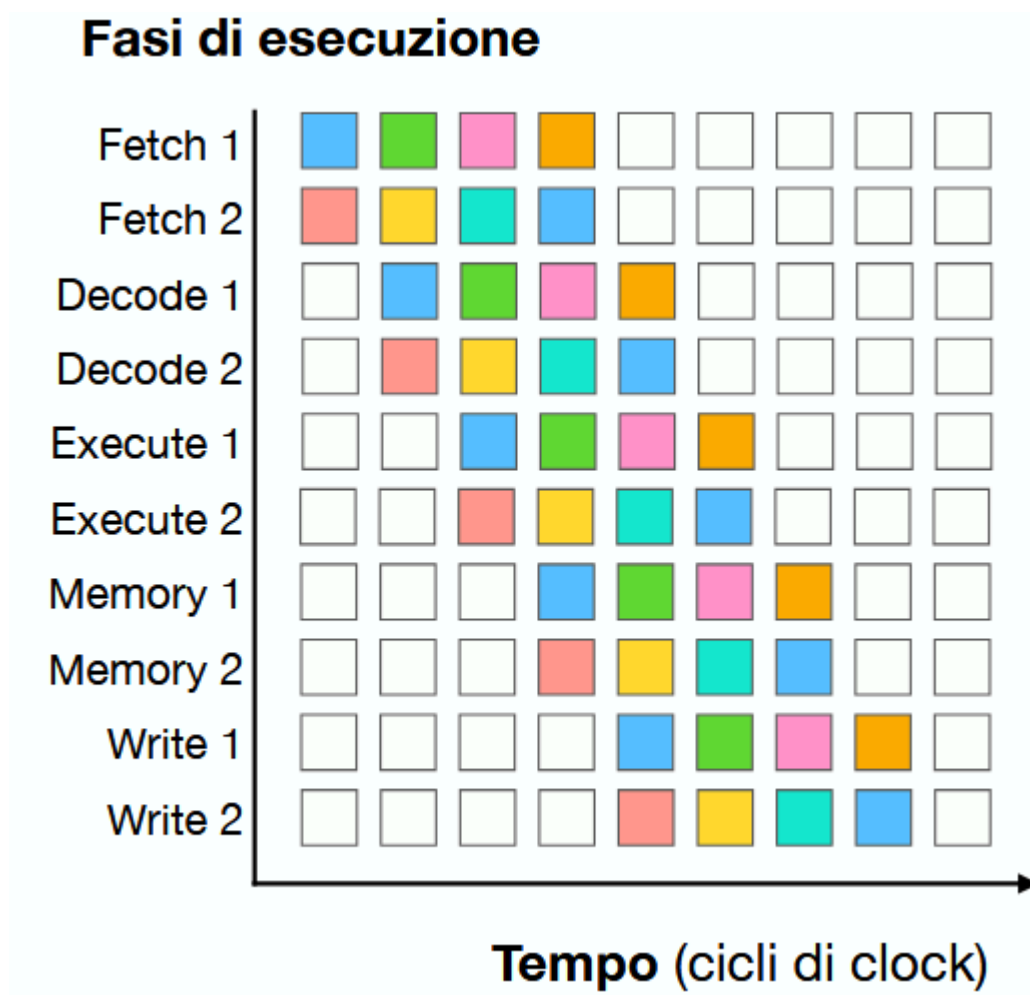
Q. Si potrebbe fare di meglio?

Sì! Adesso vediamo altre tecniche per migliorare le performance

X

3. Esecuzione Superscalare

Se abbiamo *due istruzioni* su dati diversi, possiamo eseguirli in parallelo. Quindi se abbiamo delle componenti duplicate della CPU che siano in grado di verificare che le istruzioni siano effettivamente indipendenti, possiamo eseguirli contemporaneamente e migliorare la throughput del programma.



In questo modo è possibile superare la soglia dell'IPC > 1 ; tuttavia ciò non è sempre garantito. Infatti, è possibile che:

- Ad ogni istruzione il tempo cambia (a seconda degli operandi, eccetera...)
- Alcune istruzioni sono parallelizzabili tra loro e altri no; per misurare questo si può usare il *Reverse Throughput*, e con un valore < 1 si indica che più istruzioni possono eseguire nello stesso momento. Possiamo trovare questi valori in documenti specifici chiamati *instruction tables*.

- *Structural hazard*: Quando le istruzioni richiedono la stessa parte della CPU
 - Tipicamente non si può fare nulla davanti a questa problematica
- *Data hazard*: Quando un'istruzione dipende da un operando ancora da calcolare
 - Possiamo ristrutturare la computazione per limitare questa penalità, che è data dalla latenza del "*critical path*"
- *Control hazard*: Non si conosce l'istruzione successiva da eseguire, come ad esempio i *branch* nel flusso di controllo
 - Il caso in cui perdiamo più cicli (tipicamente 15-20), come hazard è molto pesante!
 - *Esempio*. L'esempio introduttivo sul calcolo della somma (sotto una certa soglia) di vettori ordinati è un caso del control hazard.

Vedremo che per controllare l'ultimo problema si parlerà dell'*esecuzione speculativa*.

Branch Prediction

X

Branch prediction: idea preliminare, esempi di branch predictor.

X

0. Voci correlate

- Pipeline

1. Esecuzione Speculativa: Introduzione

Motivazione. Vogliamo ridurre i control hazard il più possibile, essendo che sono molto costosi!

Idea. Possiamo eseguire un *branch* chiamato "*target*" e proseguire da lì senza scrivere i risultati. Da ciò abbiamo due casi: se proseguendo la computazione abbiamo che in effetti il *target* è quello giusto, allora si confermano i calcoli e abbiamo ridotto le penalità del control hazard; invece se non è quello giusto, allora bisogna rifare i calcoli da capo.

Questa è l'*esecuzione speculativa*, ancora meglio noto come *branch prediction*. Il successo di questa tecnica dipende anche dalla "*predicibilità*" del codice, ovvero con quali probabilità si distribuiscono le branch.

X

2. Esempi di Branch Predictors

Generalmente, teniamo conto di pattern su branch presi e non presi. Elenchiamo alcuni esempi di *branch predictor* specifici:

1. *Statico*: Sceglie *sempre* un certo branch. Utili in certi casi in cui si può conoscere a priori quando si ha un certo branch, come nei cicli *for*
2. *Contatore a saturazione*: Sia x_i un contatore a n bit per il branch i -esimo, lo incrementiamo se quel branch è preso o lo decrementiamo altrimenti; naturalmente se si raggiunge la cifra "*minima/massima*", non si fa nulla. La predizione tiene conto di una certa soglia
3. *A due livelli*: Si tiene uno storico di n bit che descrive un certo pattern, in particolare indicizza il predittore apposito in una tabella.
4. *Locale*: Si usano predittori diversi per branch diversi
5. *Globale*: Si usa lo stesso predittore per tutti gli branch, generalmente utile per branch correlati tra loro
6. "*Potpourri*": Si potrebbe anche mischiare le tecniche elencate sopra
7. *Altri*: Ci sono una varietà di predittori appositi, alcuni specifici per i cicli, altri per i returns, eccetera... alcuni branch predictors si basano sulle *reti neurali*!

3. Case Study: Somma su array con threshold

Studiamo un esempio specifico, per vedere come l'esecuzione speculativa influisca sulle prestazioni di programmi.

#Esempio

Esempio 6 (somma su array con threshold).

Sia v un array con numeri casuali tra 0 e 99 e sia $p \in [0, 1]$. Si vuole trovare la somma su v di elementi che rispettino il threshold, ovvero vogliamo calcolare

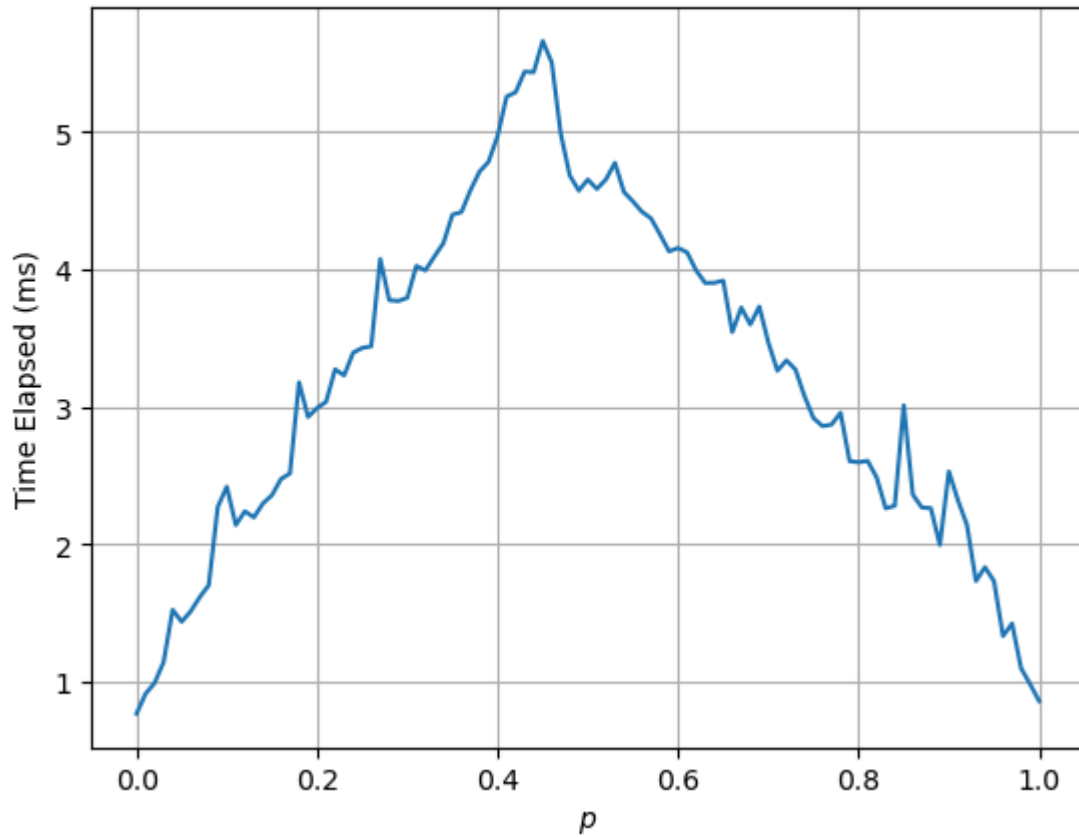
$$\sum_{i \in I} v[i], I := \{i = 1, \dots, n \mid v[i] \leq 100p\}$$

N.B. Nell'implementazione è possibile osservare che si usa la keyword `volatile`, serve per indicare che una variabile dev'essere effettivamente considerata in fase di compilazione, siccome il compilatore potrebbe automaticamente decidere di eliminare quella variabile in quanto inutile

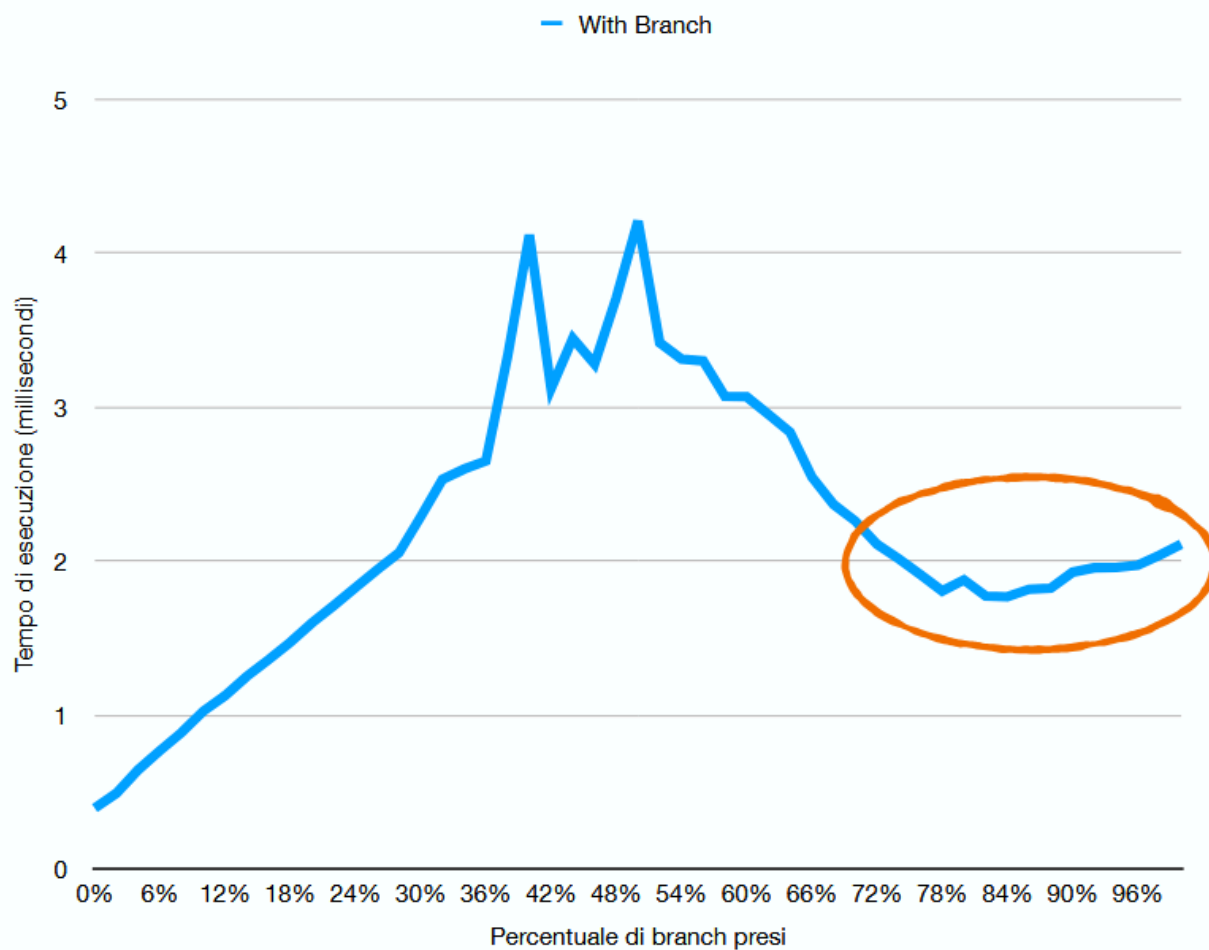
Eseguendo del codice C che risolva il problema di cui sopra - ricordiamo che ordiniamo il vettore v per poter usufruire del branch predictor! (vedere `./Lecture 03/branch_pred.c`) - e creando un grafico che indichi il tempo di esecuzione in confronto al valore p scelto, notiamo che il branch predictor ha degli effetti notevoli sulle prestazioni del programma.

In effetti, per un valore p vicino a 0.5 otteniamo che il programma ha bisogno di maggior tempo, siccome il branch predictor non riesce a indovinare in una maniera efficace il branch "giusto".

Branch Prediction - p vs time



Osservazione. In certi casi il grafico non è simmetrico in quanto per valori p maggiori si hanno più operazioni da eseguire (eventuali somme eccetera), tuttavia ciò dipende dal processore e altri fattori esterni



Programmazione Branchless

X

Programmazione Branchless. Motivazione: sostituire control hazard con data hazard. Esempio generale: condizione binaria. Study case: somma di vettore con threshold. Indicazioni generali per fare programmazione branchless (quando conviene, come fare, eccetera...). Altro case study: ricerca binaria, osservazione sull'operazione "prefetch"

X

0. Voci correlate

- Pipeline
- Branch Prediction
- Recupero dei Dati

1. Programmazione Branchless: Motivazioni ed Esempi

Per sopperire al problema di *control hazard*, si poteva ricorrere al *branch prediction* che tenta di "anullare" i costi di questa pipeline hazard tentando di "indovinare" il branch corretto. Tuttavia, non è sempre efficace in quanto potrebbe essere *"impossibile"* indovinare la scelta corretta in certi contesti.

Un'altra strada per moderare questo rischio è quello di *"sostituire"* branch in *"non-branch"*, che aggiunge più lavoro di calcolo introducendo *data hazard*, tuttavia potrebbe risultare che quest'ultimo rischio sia *"migliore"* dei branch.

Questa è la tecnica della *programmazione branchless*. Il seguente esempio motivante illustra come si *"converte"* un programma con branch in programma senza branch.

#Esempio

Esempio 7 (esempio programmazione branchless).

Supponiamo di avere due valori a, b , e una di queste viene scelta a seconda del valore booleano $q \in \{0, 1\}$. Ovvero, abbiamo:

$$x = \begin{cases} a, & q = 1 \\ b, & q = 0 \end{cases}$$

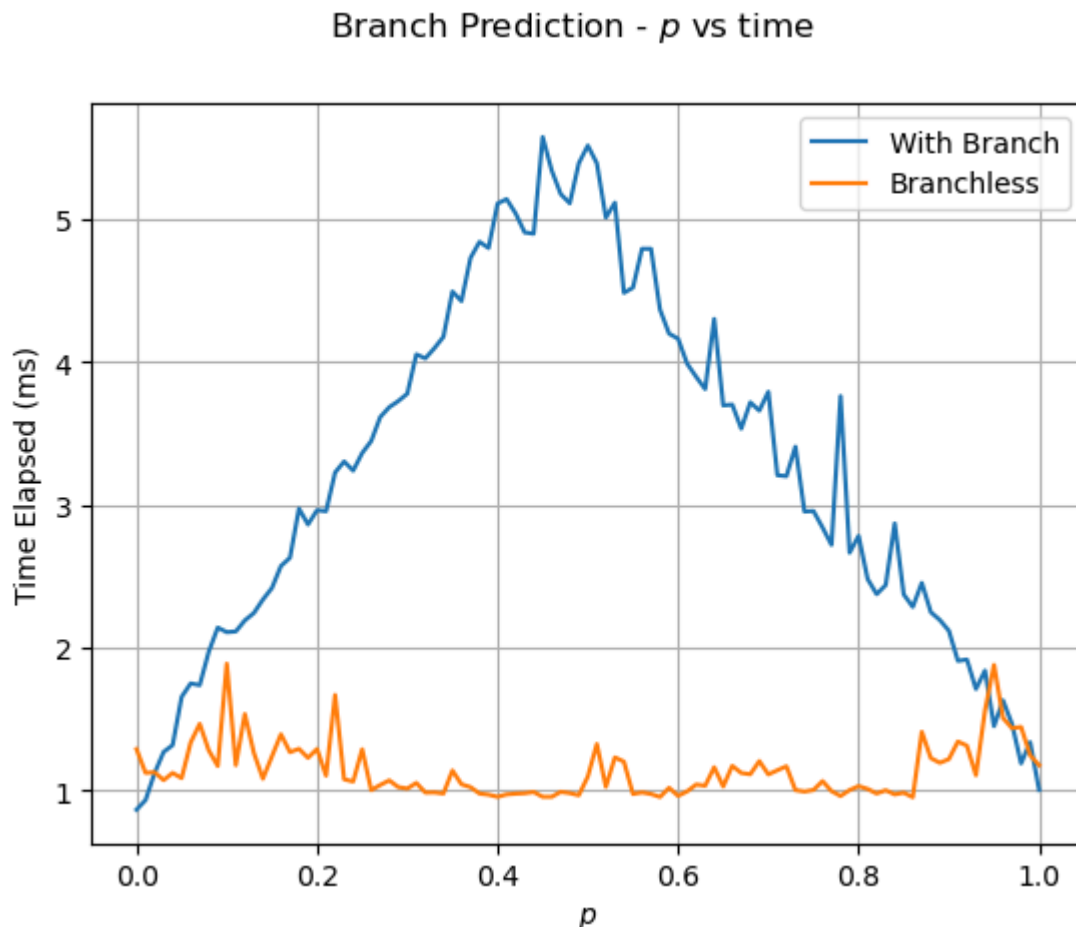
Un'implementazione *"diretta"* introduce il control hazard. Notiamo che alternativamente si può definire x nel seguente modo:

$$x = qa + (1 - q)b$$

Quindi un'implementazione diretta di questa "nuova" formula non introduce più branch, ma introduce più operazioni da effettuare, tra cui somma e moltiplicazione.

Già con questo esempio possiamo trasformare un'ampia classe di branch, in particolare quelli binari, in quanto condizioni booleane (come `x <= 10`) ritornano un valore tra 0, 1 in C.

Case Study: Applicando questa tecnica all'esempio della somma sull'array con threshold (Branch Prediction) e generando il plot dei tempi dell'esecuzione otteniamo la seguente figura.



Notiamo che In effetti "si guadagna" per valori p intermedi. Nel mio caso, si guadagna nella maggior parte dei casi (a parte le condizioni "limite"), tuttavia ciò dipende dal processore, compilatore, eccetera... (sulle slide il pezzo in cui si "guadagna" era molto più piccolo...)

X

2. Programmazione Branchless

Ancora più generalmente, si riferisce a questa tecnica come la tecnica del *predication*.

Possiamo usarla quando è possibile calcolare entrambi i lati dell'if e scegliere quale tenere solo mediante operazioni aritmetiche, e conviene quando:

- Computare entrambi i lati degli if (quindi entrambi i branch) è meno costoso che computare solo quello giusto e possibilmente sbagliare

- È difficile dire con certezza quale branch è quello giusto

Si potrebbe anche valutare di fare la *profile-guided optimization*, ossia lasciare la decisione se fare la predication o meno al compilatore. Vedremo che le istruzioni vettoriali sfruttano la programmazione branchless (con SIMD).

X

3. Case Study: Ricerca Binaria

Vediamo un altro esempio di questo paradigma, confrontandola con la programmazione branchless.

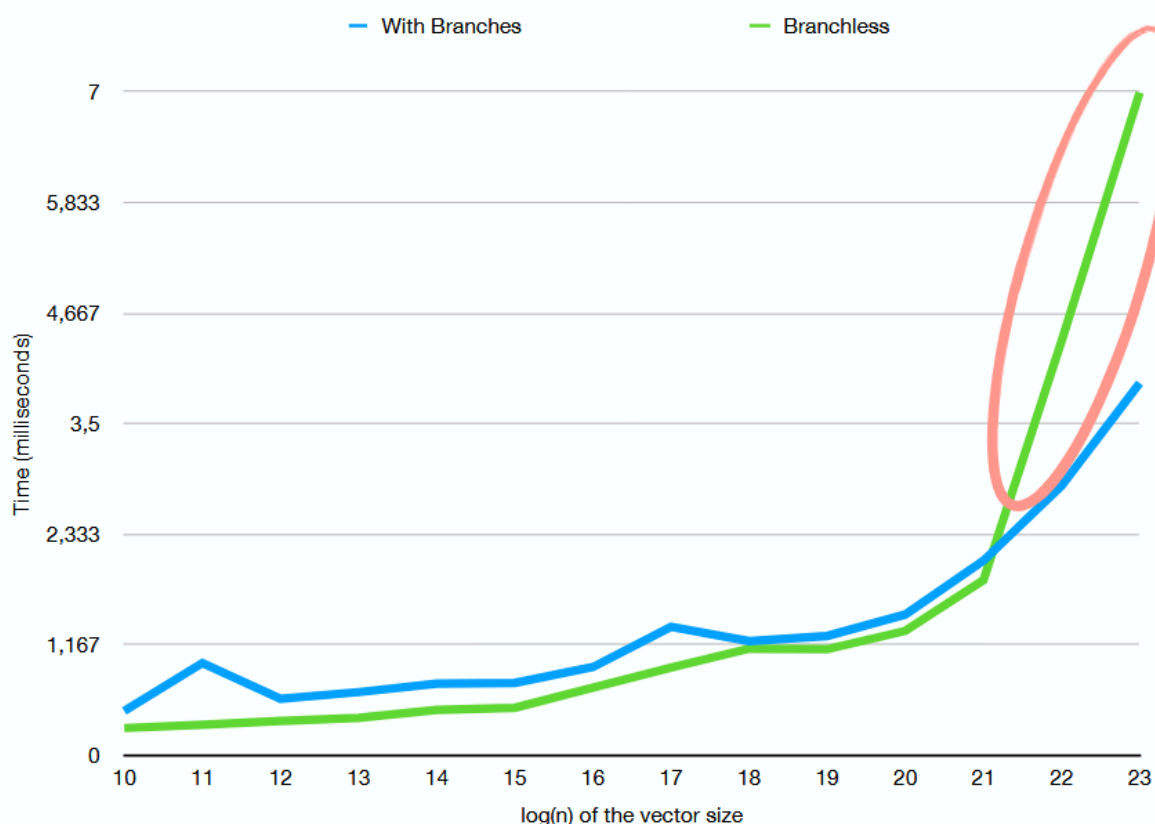
#Esempio

Esempio 8 (ricerca di dati).

Sia v un vettore ordinato di interi e supponiamo di voler cercare un elemento x . Come possiamo trovare l'indice i tale che $v[i] = x$?

Un algoritmo noto è quello della *ricerca binaria*, ossia di dimezzare l'area della ricerca per ogni passo. Sappiamo che esegue in tempo $O(\log n)$, tuttavia le costanti asintotiche dipenderanno dalla scelta dell'implementazione eseguita!

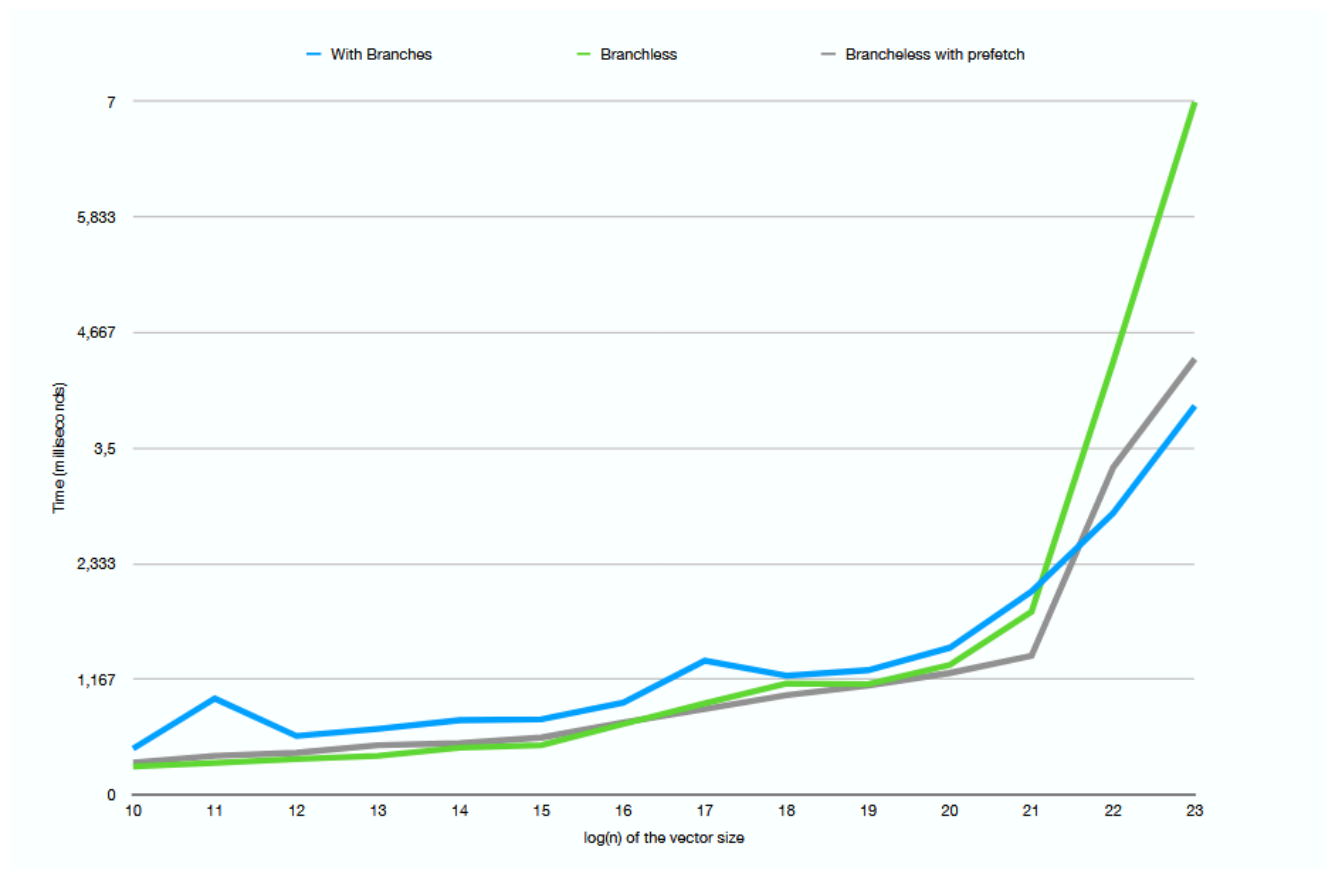
Eseguendo la ricerca binaria *sia branchless* che *con branch* otteniamo il seguente profilo dei tempi (confrontando con la dimensione del vettore) (vedere [./Lecture 04/binary_search.c](#)).



Notiamo che la versione branchless è *"leggermente migliore"*, tuttavia subisce un incremento drastico alla fine; ciò è dovuto agli effetti dell'esecuzione speculativa, infatti con l'esecuzione speculativa si comincia già a caricare i dati in memoria (in particolare $v[(l + m)/2]$ o $v[(m + r)/2]$), invece con la predication bisogna attendere di aver trovato il *prossimo indice*...

Una cosa che si può fare per risolvere questo problema è quello di dire al programma di fare il *"prefetch"*, ossia di dire al programma di caricare alcuni indirizzi di memoria che pensiamo saranno caricati successivamente. Ciò si fa con una funzionalità *"built in"* del compilatore (NON del linguaggio C!) ed è `__builtin_prefetch(void * ...)`.

Con questo otteniamo il risultato finale *"migliorato"*, che come prestazioni è un compromesso tra la programmazione con branch e la programmazione branchless:



Possiamo fare di meglio? Sì! Però vedremo in seguito...

Sequenze e Ottimizzazioni del Compilatore C

X

Due cose sul compilatore C (gcc): Sequenze della compilazione ed esempi di ottimizzazione da parte del compilatore.

X

0. Voci correlate

- Opzioni Comuni del Compilatore C
- Extern e Header
- Creazione delle Librerie in C

1. Sequenza della Compilazione in C

Ricordiamo la sequenza della compilazione di un programma in C:

1. *Preprocessing*: Espando le *macro* (ovvero ciò che segue il simbolo del cancelletto `#`), rimuovo i commenti, inserisco gli header, eccetera...; per farlo "manualmente" si scrive il comando `gcc -E ...`
2. *Compilazione* (`gcc -S ...`): La compilazione effettiva in linguaggio assembly, quindi in realtà di tratta di un passaggio intermedio.
3. *Assembly* (`gcc -c ...`): Da assembly si converte il codice in codice macchina eseguibile; notiamo che non è il passaggio finale, in quanto ci sono comunque i *placeholder* che richiamano funzioni o strutture esterne
4. *Linking* (`gcc -o ...`): Passaggio finale, genera il file eseguibile sostituendo i placeholder

Una cosa interessante che possiamo fare per analizzare il codice C è quello di osservare l'*assembly* generato dal codice, da questo possiamo dedurre quali tipi di operazioni otteniamo e dedurre se ci sono stati delle ottimizzazioni in mezzo.

Si può passare al compilatore l'argomento `-fverbose-asm` per ottenere del codice assembly commentato, che descrivono le corrispondenze riga-per-riga tra codici. Un altro modo per farlo è quello di usare Godbolt o Compiler Explorer.

X

2. Ottimizzazioni del Compilatore C

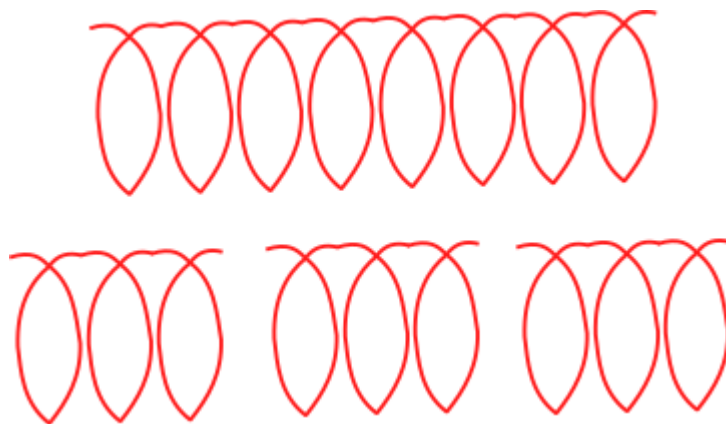
Vediamo una carrellata di esempi di ottimizzazioni fatte da parte del compilatore di C.

Constant Propagation. Se possiamo calcolare un valore direttamente allora posso inserire direttamente il risultato durante la compilazione. Per esempio, se ho la variabile $x = 2 + 3$ che

viene calcolata come $x = 5$, posso sostituire x con 5 nel codice compilato.

Dead-code Elimination. Si tratta di rimuovere del codice "ininfluente" sull'output, come ad esempio variabili che non verranno stampate o parti del codice mai eseguite

Loop Unrolling. "Srotoliamo" un ciclo (di cui conosciamo il numero di iterazioni a priori) in singole iterazioni; ciò è utile in quanto possiamo diminuire il numero di branch. Tuttavia ciò comporta un trade-off tra la memoria richiesta e la velocità del programma, quindi solitamente si fa uno "srotolamento" parziale.



Ad esempio, possiamo sostituire un ciclo di 1000 iterazioni di tipo `v[i]++` con 100 cicli di iterazioni di `v[i]++; v[i+1]++; ...; v[i+9]++;`.

Si può giostrare con questa ottimizzazione con le seguenti opzioni o direttive del compilatore:

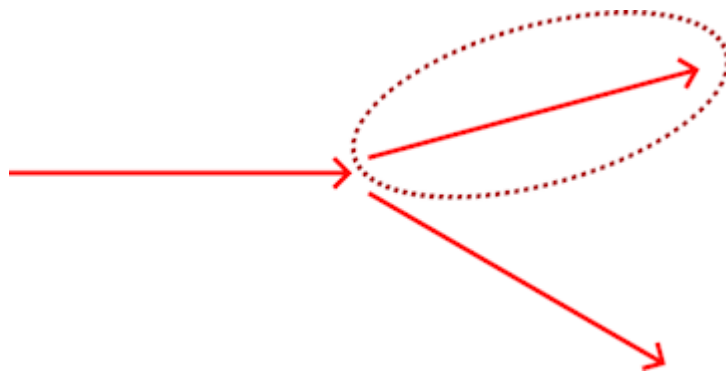
- Con l'opzione `-funroll-loops` possiamo dire al compilatore che *può* effettuare questa ottimizzazione (ma non è detto che lo faccia!)
- Con la direttiva `#pragma GCC unroll n` per specificare "quanto" fare il srotolamento

Function Inlining. Si sostituisce una chiamata ad una funzione col corpo della funzione; con questa evitiamo di dover far chiamate a funzioni, tuttavia ciò comporta l'espansione del codice in quanto duplichiamo le funzioni.



Solitamente conviene quando la funzione è piccola, e si può *suggerire* (non forzare!) al compilatore di farlo con la keyword `inline` prima della definizione di una funzione.

Branch Likelihood. Si può indicare al programma quale branch avrà maggiore probabilità di essere preso (conoscendolo per qualche strano motivo), lo si fa col metodo built-in `__builtin_expect(expr, val)`, dove `expr` è un'espressione da valutare e `val` è il risultato atteso.



Ci sono anche molti altri tipi di ottimizzazioni (almeno una decina), alcuni si basano su delle euristiche, alcuni possono essere abilitati o disabilitati con opzioni del compilatore, eccetera...

Cache

X

Gerarchie di memoria: la cache, motivazioni e principi. L'accesso della cache ai dati, principio di cache line, associazione 1-way, full e k-way.

X

0. Voci correlate

- Approcci Storici per l'Organizzazione della Memoria
- Introduzione alla Programmazione Avanzata e Parallela

1. La Cache

Vediamo il tempo richiesto per eseguire alcune operazioni del processore:

Tempo	Operazione	Note
0.3ns	ADD, OR, SUB, ...	
0.5ns	Cache L1 dati	
1ns	La luce percorre ~30cm	Ci si aspetta rimanga costante
5ns	Branch misprediction penalty	
5-7ns	Cache L2	~2020
10ns	DIV	
19ns	Cache L3	
100ns	Mutex Lock/Unlock	
100ns	Accesso alla memoria principale	~2020. Sono 200-300 cicli

Osservazione. L'operazione aritmetica di somma/sottrazione e simili richiedono lo stesso tempo per cui la luce viaggia 10 centimetri. Invece per l'accesso alla cache sono 15 centimetri, e alla RAM 3000 centimetri! Quindi è molto importante trattare la gerarchia delle memorie.

Per un motivo analogo è molto difficile rendere la memoria più veloce, ovvero si tratta di una questione della propagazione del segnale e della densità; infatti, basta immaginarsi un intorno attorno alla ALU di 10 centimetri, ed è ciò che può viaggiare la luce in 0.3ns. L'unico modo per rendere la memoria più veloce è quella di avvicinarla alla ALU il più possibile, ovvero compattarlo il più possibile.

Quindi si aggiunge una cache oltre all'unità di memoria principale per mantenere i dati più recenti a cui abbiamo acceduto; in particolare, se la cache è più rapida della memoria principale allora

ogni accesso ad un indirizzo visitato sarà ancora più rapido. Per sfruttare bene questo effetto, la cache si basa sul principio di località:

Il principio di località

Il principio di località consiste in due parti:

La *località temporale*, ovvero vogliamo accedere ad un indirizzo di memoria in un range di tempo

La *località spaziale*, ovvero se accedo ad un indirizzo di memoria accedo anche i suoi eventuali successori (o predecessori)

Quindi se un programma rispetta bene il principio della località, le cache riescono ad essere efficaci nel migliorare la prestazione.

Ricordandoci che c'è il trade-off tra la velocità e la dimensione dell'unità di memoria, abbiamo più livelli di cache:

- L1: La più veloce, divisa in cache dei dati e delle istruzioni
- L2: Per un core singolo
- L3: Per più core
- L4: Disponibile solo in alcune architetture (in particolare quelle recenti)

X

2. Accesso della Cache ai Dati

Q. Come accede la cache ai dati? In particolare, come si può fare in un modo tale da poter sfruttare il principio di località?

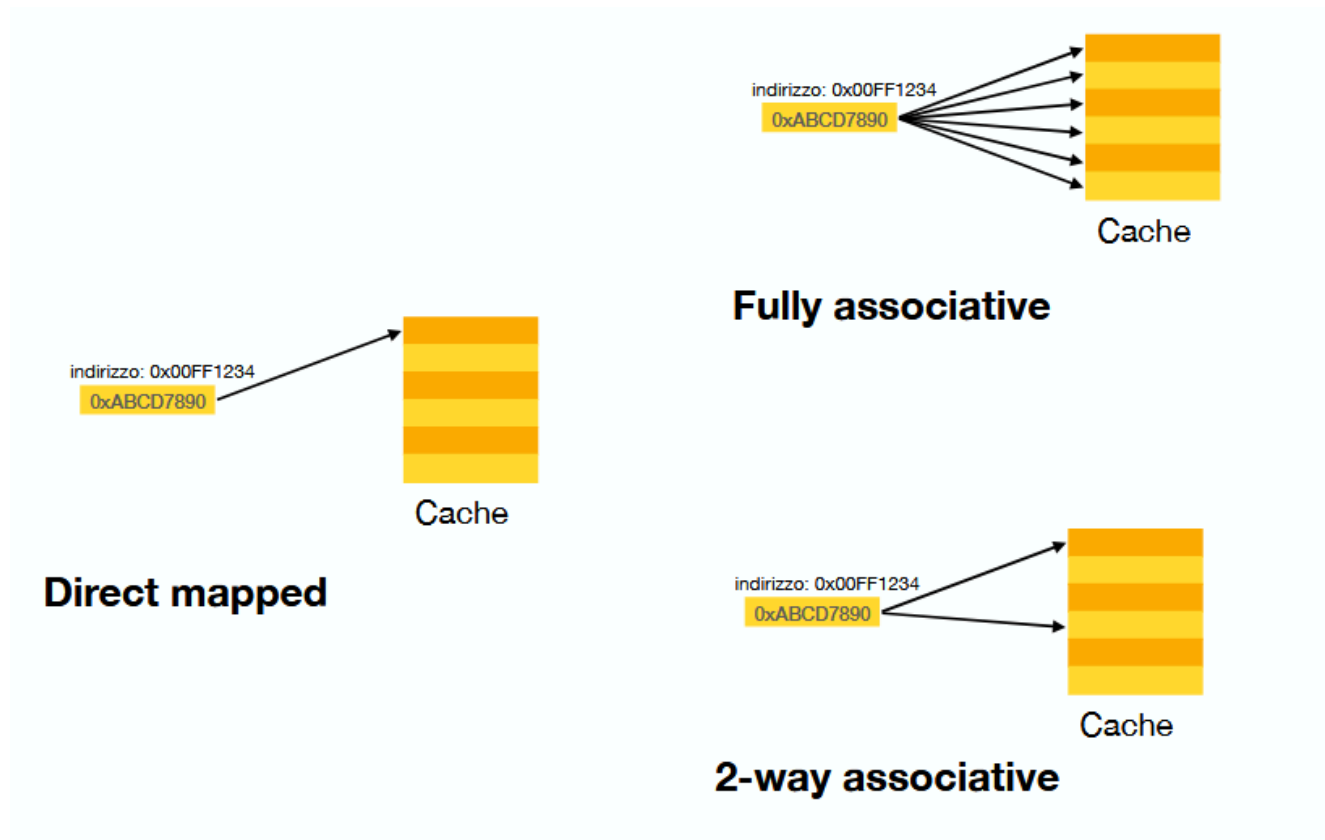
Ha senso caricare dei *blocchi di memoria*, che nel gergo tecnico si chiamano *cache line*. Solitamente consistono in blocchi continui da 64 o 128 byte. In questo caso, sfruttiamo la località spaziale: infatti, una volta fatto un accesso, l'accesso agli altri indirizzi della stessa cache line sarà "gratis"!

Inoltre bisogna poter associare un indirizzo ad una specifica linea di cache, ci sono più approcci:

- *One-way*: Ogni indirizzo può essere un solo "*slot*". Se due indirizzi condividono lo stesso slot, allora la vecchia cache line sarà rimossa e rimpiazzata da quella nuova.
- *Fully Associative*: Ogni cache line può essere inserita in ogni "*slot*", se tutte le posizioni sono occupate allora bisogna rimuoverne una (la "*replacement policy*"). Un esempio di *replacement policy* è la *Least Recently Used*, ovvero la cache line a cui è stata acceduta più indietro nel tempo.

Con l'approccio *one-way* abbiamo un'implementazione rapida ma poco efficiente in termini di "*hit rate*", ovvero la probabilità di trovare l'indirizzo richiesto nella cache. Simmetricamente l'approccio *fully associative* ha una buona hit rate ma è più difficile da implementare e rendere veloce.

Un compromesso tra gli due approcci è la *k-way associative*, ovvero ogni indirizzo può essere associato a k slot.



Disposizione in Memoria

X

Disposizione in memoria: come le strutture dati influenzano le prestazioni dei programmi. Liste concatenate, Unrolled linked list. Effetto dell'allineamento delle strutture (in C), esempi.

Osservazione: l'effetto padding. Case study: Array di Strutture e Strutture di Array.

X

0. Voci correlate

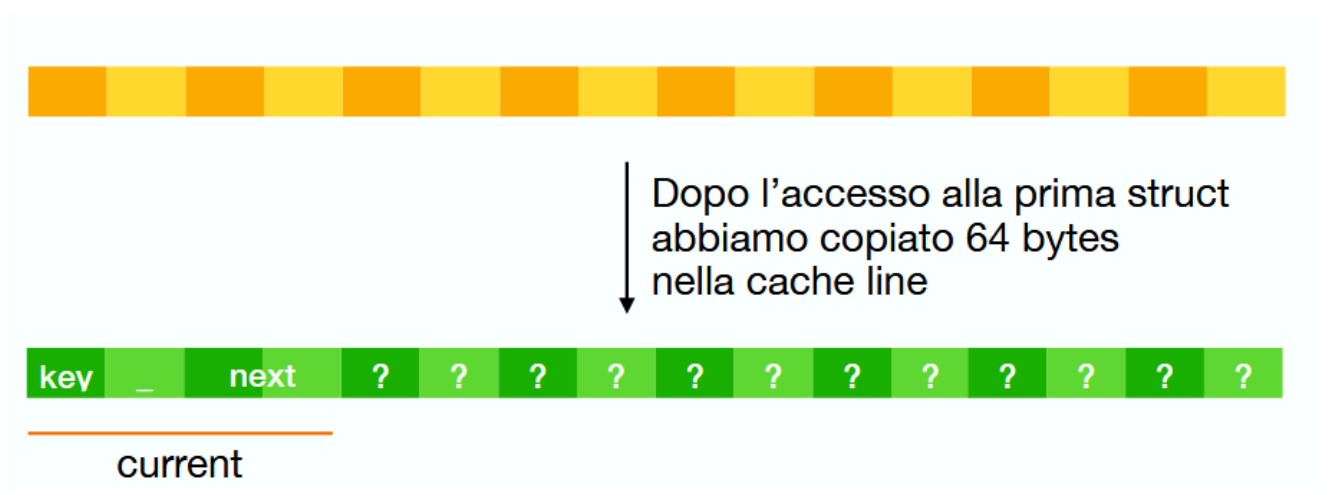
- Cache di un Processore
- Introduzione alla Programmazione Avanzata e Parallela
- Liste e Array

1. Disposizione in Memoria

Vediamo le *conseguenze* del fatto che per sfruttare al meglio la cache si necessita di strutture dati "*locali*", ovvero che rispettano il principio della località. In altre parole, vediamo come le strutture dati influenzano le prestazioni degli algoritmi.

Linked List. Ricordiamo che una *linked list* ("lista concatenata") è una struttura che consiste in due elementi: un valore intero (o il dato che ci interessa) e un indirizzo che punta al prossimo elemento della lista concatenata.

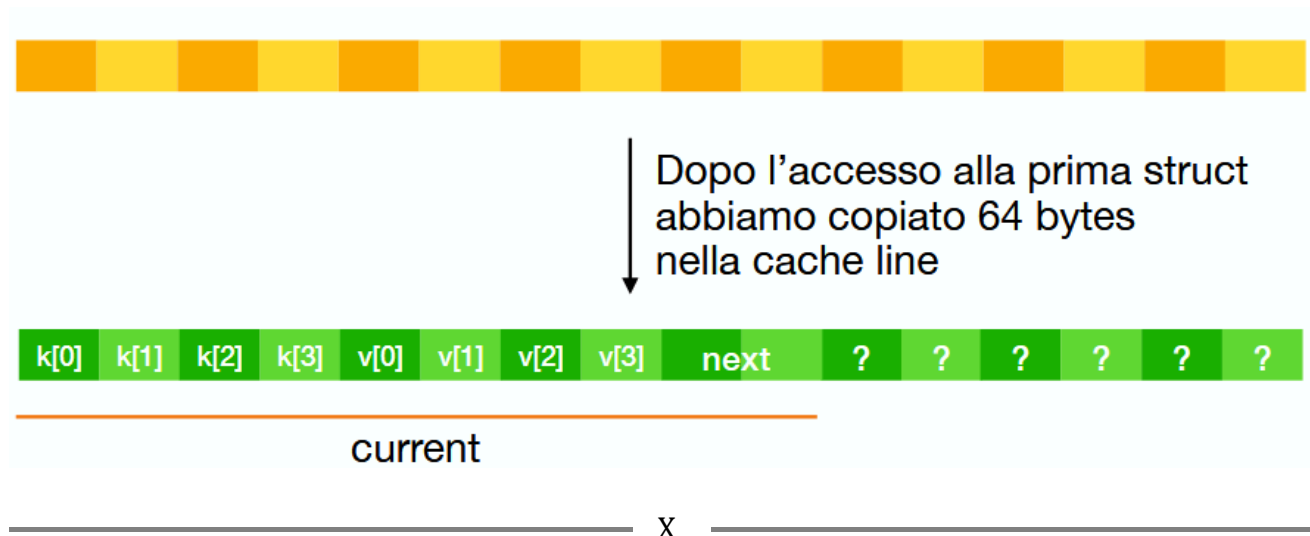
Questa struttura ha una *pessima* località di memoria, infatti i nodi sono "*sparsi in giro*" in quanto ci serve l'elemento "*next*" (ovvero il puntatore) per trovare l'elemento successivo. Per ogni cache line leggiamo *solo* un valore.



Unrolled Linked List: Adesso abbiamo a che fare con una struttura dati che contiene un *vettore* (di dimensione fissata N) come dato. Da un punto di vista implementativo ci serve un vettore secondario per indicare quali elementi siano "*validi*" (nel senso che hanno un valore definito) o

meno. Poi come nelle liste concatenate, abbiamo un puntatore che indica l'indirizzo dell'elemento successivo.

Dal punto di vista della località è leggermente migliore, siccome con una cache line carichiamo più valori, quindi pago la penalità solo una volta per nodo, ossia per ogni N valori.



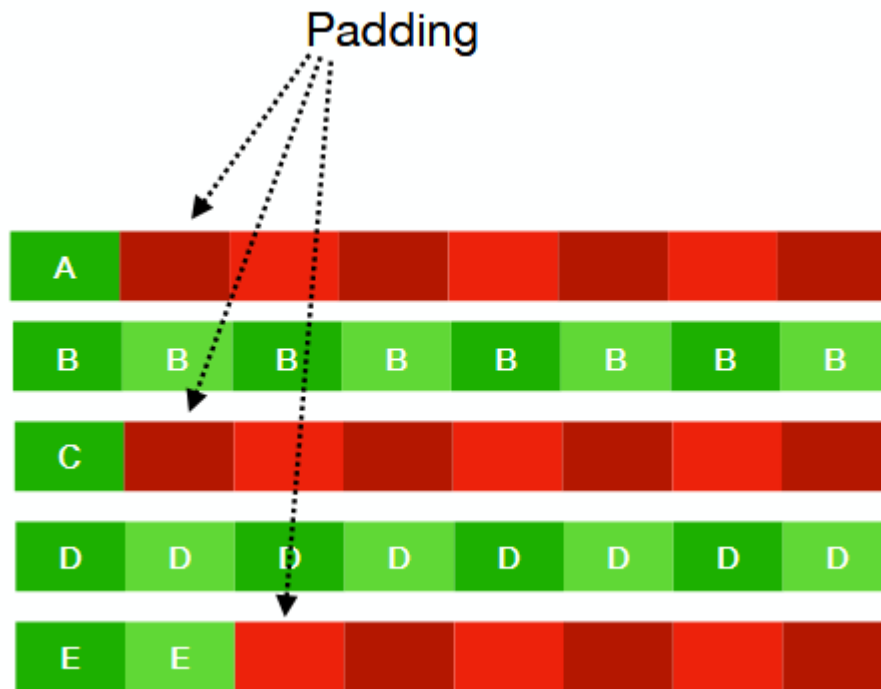
2. Allineamento delle Strutture

Osservazione. Solitamente i processori accedono a memoria in blocchi di multipli di 4 o 8 byte. Per avere un accesso alla memoria efficiente, il compilatore allinea i membri (in ordine di definizione) di una struttura in un modo che sia possibile farci l'accesso in modo allineato. Per farlo si aggiunge del *padding*, ossia si lascia dei "byte vuoti" per poter realizzare questo allineamento. In questo modo, per leggere il valore di un campo di una struttura la si fa con il minor numero di caricamenti di indirizzi possibile.

Negli seguenti esempi, supponiamo che il processore sia in grado di caricare 8 byte alla volta.

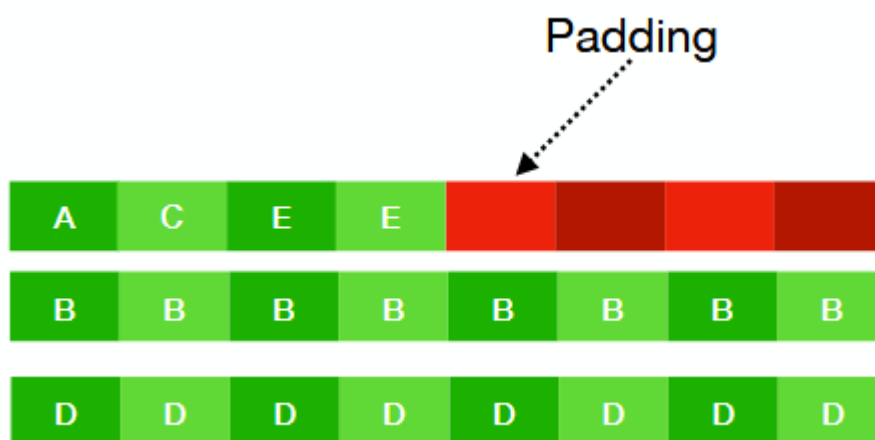
Caso 1. Consideriamo la struttura `struct foo { int8_t a; int64_t b; int8_t c; int64_t d; int16_t e };`. Teoricamente ogni struttura contiene 20 byte di informazioni effettivi, tuttavia chiamando la funzione `size` sulla struttura notiamo che occupa 40 byte, invece di 20; come mai?

Questo è dovuto all'effetto dell'allineamento, infatti si effettua del *padding* su alcuni campi per poter rendere altri campi accessibili in un singolo caricamento. Quindi abbiamo dello spazio sprecato...



Osservazione. Si aggiunge padding sull'ultimo membro così da poter effettuare l'allineamento alla prossima struttura.

Caso 2. Consideriamo adesso la struttura `struct foo { int8_t a; int8_t c; int16_t e; int64_t b; int64_t d };`. Notiamo che in memoria viene rappresentata nel seguente modo:



In particolare, occupa meno spazio in quanto c'è bisogno di fare *meno* padding.

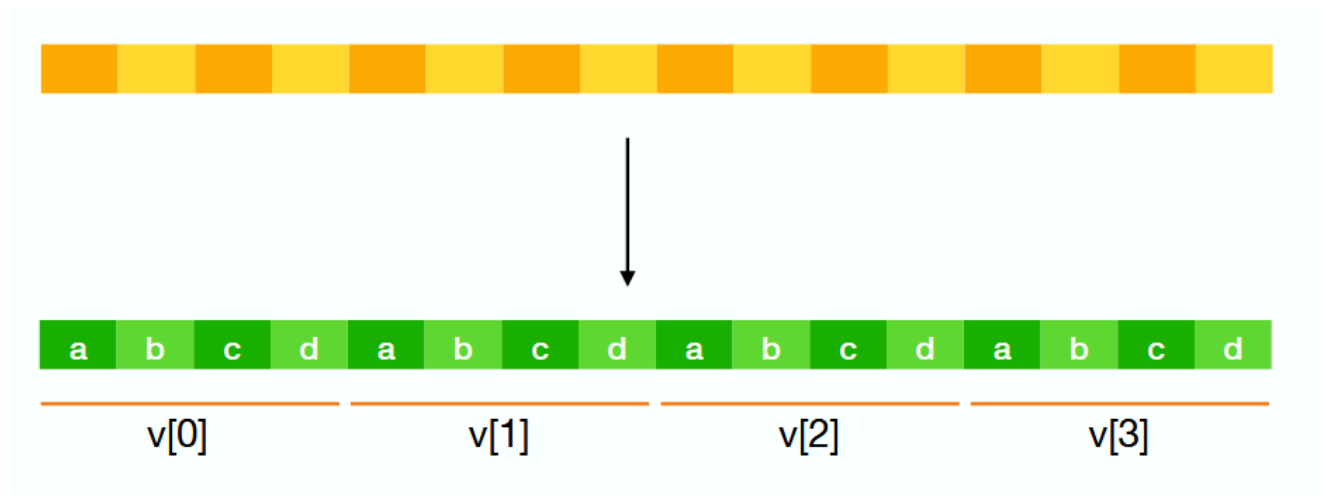
Nota. Se si volesse, per qualche motivo come ad esempio rispettare alcuni vincoli delle librerie, impedire il padding a discapito delle prestazioni allora si usa il metodo *non standard* `__attribute__((packed))` nella definizione della struttura (prima del nome).

3. Case Study: Strutture di Array e Array di Strutture

Array di Strutture. Sia `foo` una struttura con 5 valori interi, che occupano quindi 4 byte ciascuna e sia `v[n]` un'array che contiene più strutture `foo`.

Supponiamo di voler calcolare la somma dell'array su `foo.a`. Quali sono gli effetti sulla cache?

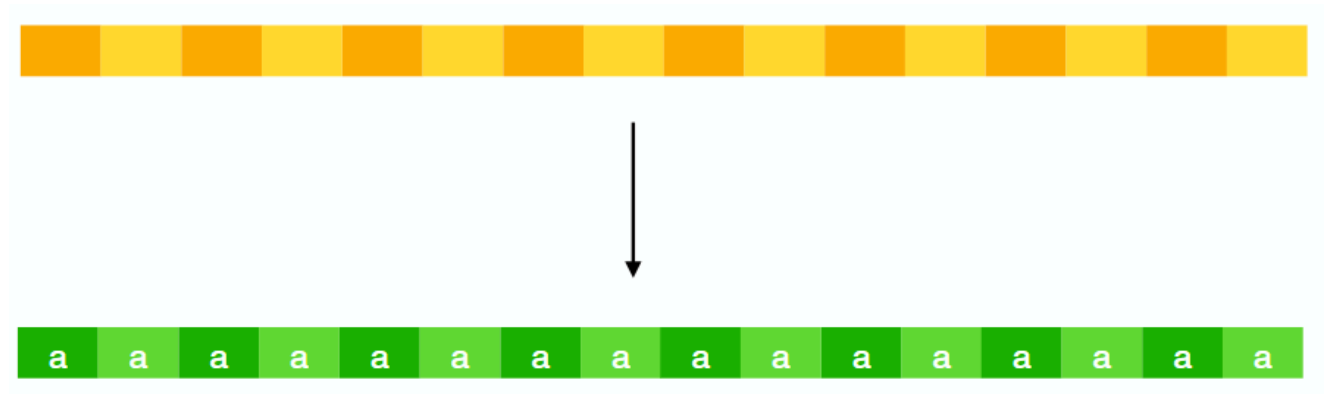
Carichiamo la struttura comprendendo tutti i 5 campi, tuttavia ne leggiamo solo una parte; quindi in un certo senso non abbiamo sfruttato bene la cache, in quanto tendiamo a "sprecare" delle cache line.



Possiamo fare di meglio? Sì! Vedremo come: l'idea guida consiste in invertire l'array di strutture in una struttura di array.

Strutture di Array. Sia `bar` invece una struttura di array, ovvero contiene cinque array che contengono i valori desiderati.

Considerando lo stesso problema di prima, notiamo che sfruttiamo la località molto meglio in quanto con *una sola cache line* carichiamo il numero di valori del campo `a` il più possibile.



Conclusione. Array di strutture (AoS) e strutture di array (SoA) sono due strutture dati "equivalenti", che differiscono nel pattern di accesso ai dati. Nelle AoS accedo a tutti i campi volta per volta, invece negli SoA itero su un singolo campo (o più). Tuttavia tendenzialmente le AoS sono più facili da leggere, quindi si usano le SoA solo quando sono indispensabili per le prestazioni di un programma (come nel problema di prima).

Località della Ricerca Binaria

X

Case study della località in memoria: la ricerca binaria. Analisi della località temporale e binaria della ricerca binaria "standard", osservazioni. Ricerca binaria alternativa: idea preliminare, costruzione dell'array alternativo. Recap di come velocizzare algoritmi tenendo conto della località di memoria e di altri fattori.

X

0. Voci correlate

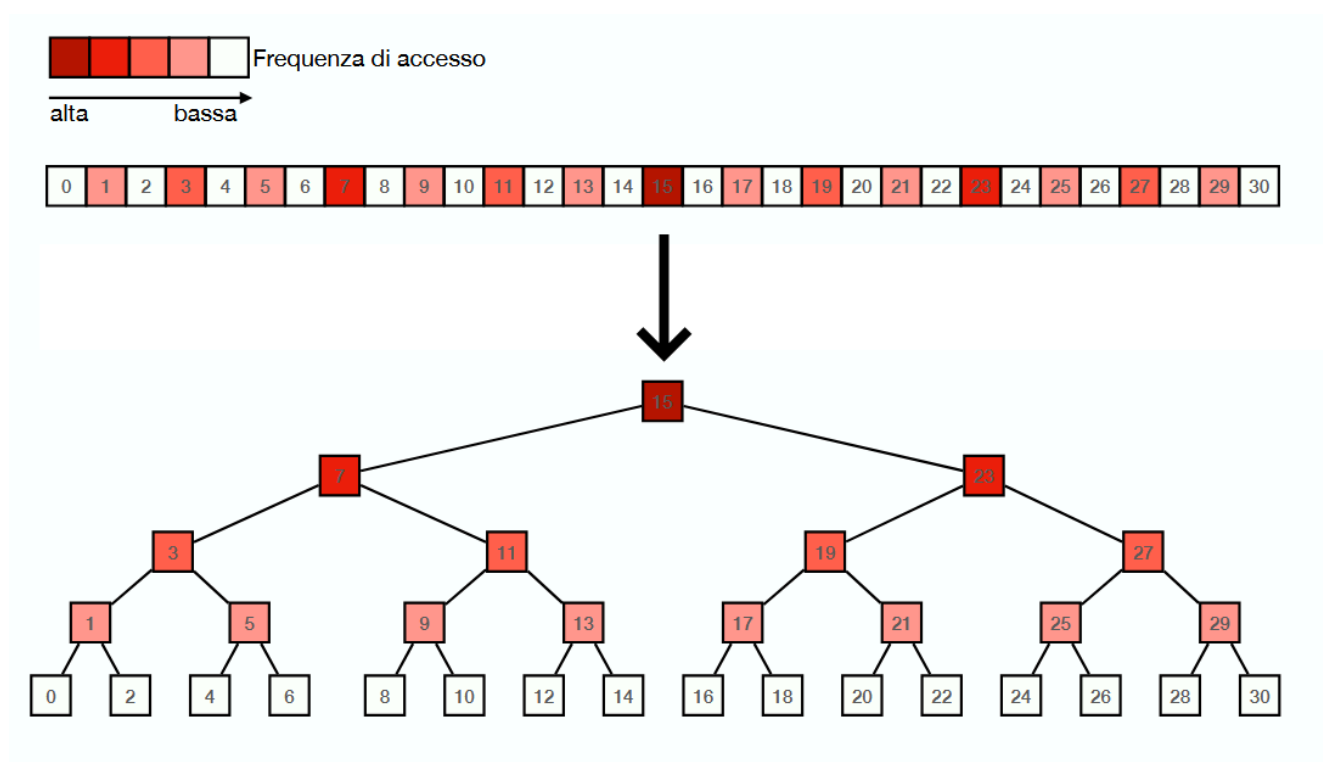
- Cache di un Processore
- Disposizione in Memoria
- Recupero dei Dati

1. Ricerca Binaria "Array"

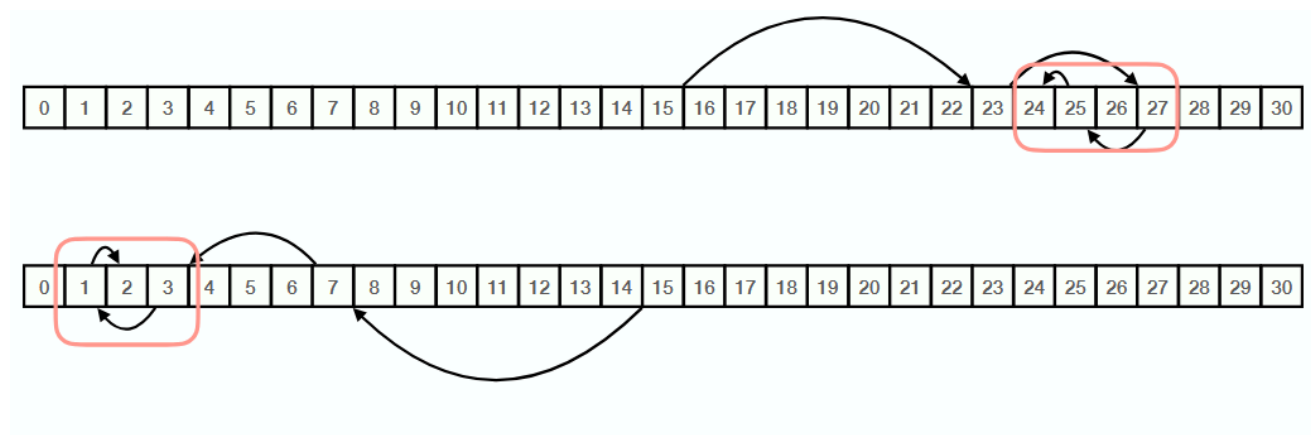
Q. Quali sono le proprietà di località della ricerca binaria?

Località temporale: Se facciamo *molte* ricerche binarie sullo stesso array, notiamo che si tende ad accedere gli elementi di mezzo, di quarto, di ottavo, eccetera.. più frequentemente. Notiamo che quindi possiamo disegnare un albero decisionale, in cui la radice parte dall'elemento di mezzo e le ramificazioni consistono in considerare la metà dei due sottoarray disegnati dall'elemento di mezzo.

In questo modo, vediamo ancora più evidentemente la distribuzione degli accessi nel tempo



Località spaziale: Osserviamo i pattern di accesso: all'inizio gli intervalli che consideriamo sono *larghi*, però man mano alla fine gli intervalli diventano più piccoli.



Recap. Quindi:

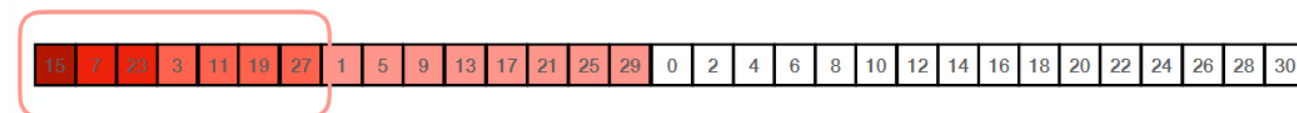
- All'inizio si ha una *buona località temporale* ma una *pessima località spaziale*
- Viceversa alla fine, ossia *pessima località temporale alla fine* ma *buona località spaziale all'inizio*.

Possiamo fare di meglio? Sì! Vediamo adesso come.

X

2. Ricerca Binaria "BST"

Idea. Avevamo osservato che possiamo immaginare la ricerca binaria come la visita di un albero. Quindi riordinando opportunamente l'array originario possiamo effettivamente fare una ricerca su un albero invece su di un array, possibilmente migliorando le proprietà di località.



Con questa soluzione, si avrebbe che le proprietà di località sarebbero più allineate, infatti all'inizio l'algoritmo gode sia una buona proprietà di località spaziale e temporale, siccome la parte dell'albero vicina alla radice viene subito messa tutta in cache. Poi eventualmente le proprietà di località peggiorano lo stesso...

Vediamo come trasformare l'array ordinato in una rappresentazione dell'albero binario di ricerca appena descritto.

ALGORITMO. ("Sorted to BST")

- Sia $A[n]$ un array ordinato di n elementi

- Sia $T[n + 1]$ l'array target, che sarà l'output dell'algoritmo. Notiamo $n + 1$: per semplificare l'implementazione dell'algoritmo non considereremo mai $T[0]$.
- Siano $i = 0, k = 1$ indici su A, T .
- Effettuare ricorsivamente la seguente procedura:
 - Scrivere ricorsivamente i valori (a partire da i) a partire da $2k$ a sinistra di quello di mezzo aggiornando i
 - Scrivere il valore di mezzo (dato dal nuovo valore di i)
 - Scrivere i valori rimanenti a partire dalla posizione $2k + 1$

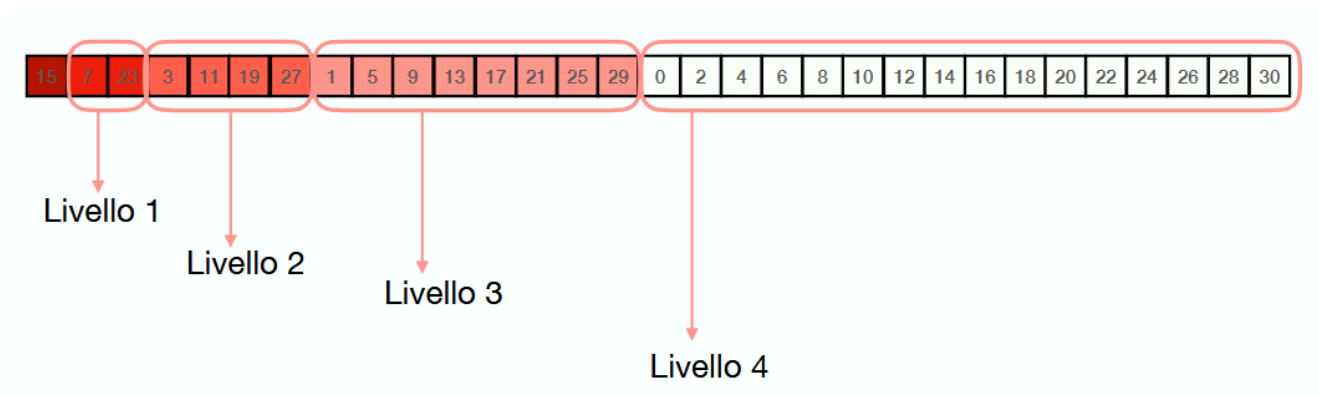
(??? non ho capito sinc)

Poi per fare la ricerca binaria sul nuovo array basta modificare opportunamente l'algoritmo, in particolare:

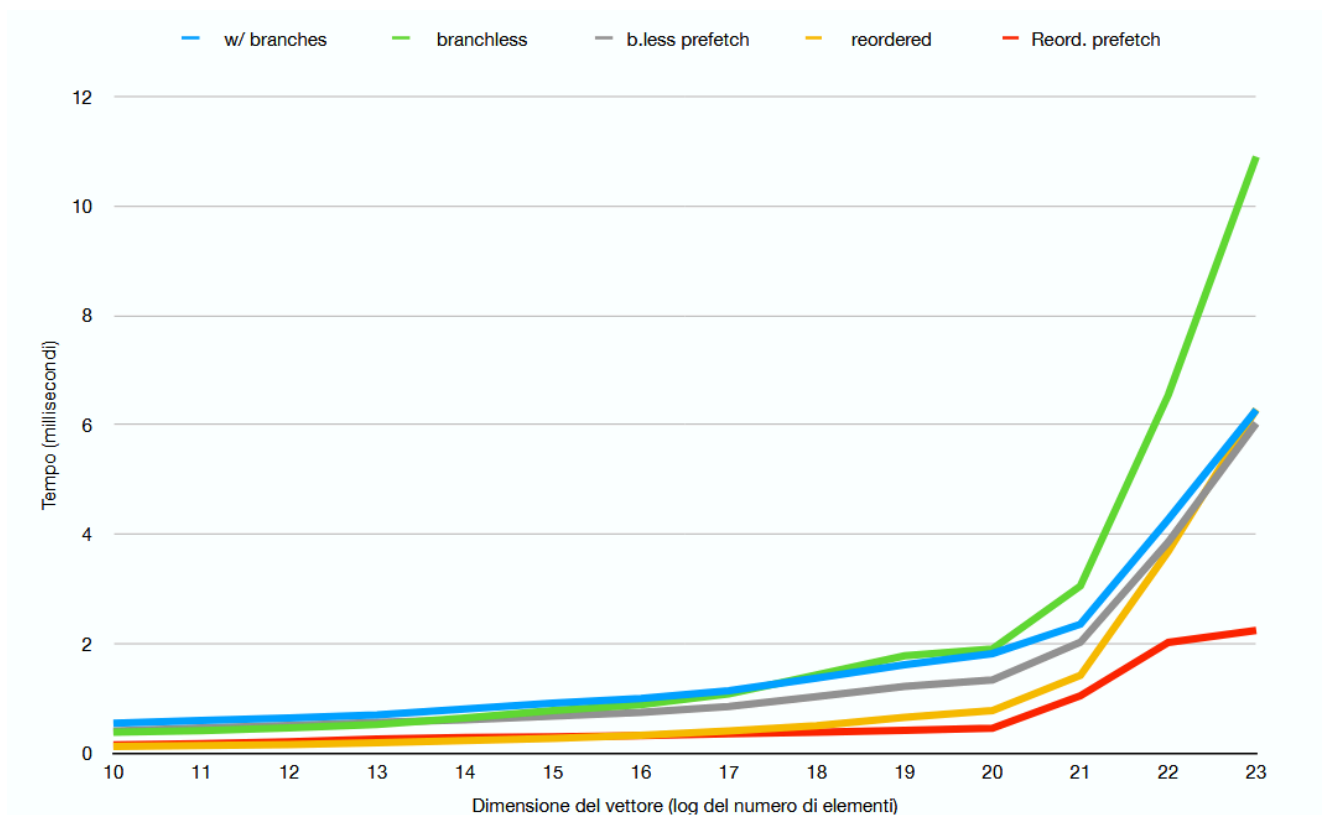
- La ricerca parte da $k = 1$
- Se la chiave è minore allora raddoppiamo l'indice ($2k$)
- Se è maggiore allora raddoppiamo e aggiungiamo 1 ($2k + 1$)
- Se è uguale allora OK

Osservazione. Dato l'indice k di un elemento, sappiamo dove saranno i suoi figli e sicuramente accederemo al prossimo livello. Per k elevato non saranno nella stessa cache line in cui si trova ancora l'elemento corrente: quindi per migliorare l'algoritmo si potrebbe fare un prefetch di quel blocco!

Generalmente, se siamo al livello i allora nel livello $i + 4$ (limitato alla parte che possiamo raggiungere) ci sono 2^4 elementi consecutivi in memoria.



Abbiamo i seguenti risultati:



Conclusione. Posso velocizzare un algoritmo facendo le seguenti cose:

- Valutare la programmazione branchless, dove possibile e dove conviene
- Considero le interazioni con la memoria, soprattutto quando la dimensione supera quella della cache
- Sapere quali indirizzi mi serviranno per effettuare i prossimi passi dell'algoritmo
- Tenere conto le proprietà di località

Richiami all'interazione con i files in C

X

Breve descrizione qui

X

0. Voci correlate

- Libreria Standard per File

1. Accesso a Files in C

Per accedere ai file in C si usano le *system call* fornite dal sistema operativo, che ci dà una struttura con una specie di "cursore" sul file e ci permette di accedere al contenuto del file come se fosse un flusso (*stream*) di caratteri/byte.

Vediamo una carrellata (ripasso) di funzioni della libreria standard per interagire con i file salvati in memoria di massa (HDD o SSD).

1. `FILE * fopen(char * path, char * mode);` - Ci ritorna un puntatore ad una struttura "FILE" (non ci interesserà l'implementazione)
2. `void fclose(FILE * fp);` - Chiudere il file, ovvero per liberare le risorse associate e svuotare tutti i buffer associati
3. `fprintf(FILE * fp, char * format, ...); fscanf(FILE * fp, char * format);` - Scrivere sul file o leggere caratteri dal file seguendo uno formato specifico
4. `fgetc, fgets, fputc, fputs` - Anche questi per scrivere o leggere dal file, però le funzioni agiscono ad un livello più basso.
5. `int fread(void * ptr, size_t size, size_t nmemb, FILE * fp); int fwrite(...)` - Come prima, ma scrivono dei "dati grezzi" invece di caratteri; utile per scrivere su file binari.
6. `int fseek(void); int fseek(FILE * fd, long offset, int whence);` - Per interagire col "cursore" sul file aperto, il primo caso per ottenere la posizione e secondo caso per modificare la posizione.

Ricordiamo inoltre il sistema del *buffer*, ovvero in realtà il sistema operativo non scrive direttamente sulla memoria di massa, ma tiene un buffer in cui tiene una quantità di caratteri letti o da scrivere che poi viene svuotata quando si riempie o quando è passato abbastanza tempo.

Questo sistema serve per minimizzare le operazioni di scrittura e di lettura, che sono costose! In un certo senso, abbiamo lo stesso principio della cache; solo che qui la RAM agisce da cache nei confronti della memoria di massa.

Modello con Memoria Esterna

X

Modello del processore con memoria esterna: modellare i costi delle interazioni input/output, idea di ordini di grandezze. Esempi di analisi: Array Scan e strutture ad albero.

X

0. Voci correlate

- Richiami all'interazione coi files in C
- Introduzione alla Programmazione Avanzata e Parallela

1. Modello del Processore con Memoria Esterna

Motivazione. Tradizionalmente si conta ogni operazione con costo unitario quando calcoliamo il costo asintotico di un algoritmo. Abbiamo già visto che in realtà quando accediamo alla memoria principale questa è solo un'approssimazione; l'approssimazione diventa ancora meno vero quando dobbiamo accedere alla memoria di massa, siccome le operazioni I/O con questa parte è 100-1000 volte più lenta.

Infatti, per darci un'idea vediamo la seguente tabella dei costi "veri", comparata ad una versione "normalizzata" in secondi.

Memory	Time (real)	Time (normalized)
L1 Cache	2 ns	1 s
RAM	0.00001 ms	50 s
SSD	0.1 ms	13 h, 52 m
HDD	10 ms	57 d, 20 h, 52 s

Pertanto ci serve un modello *alternativo* della modellazione dei costi, per tenere conto delle operazioni *input/output*.

Modello.

Consideriamo N la dimensione dei dati su cui dobbiamo lavorare che è suddiviso in blocchi di dimensione B elementi. Dopodiché assumiamo che nella RAM possiamo salvare al massimo M elementi, quindi possiamo tenere solo $\lfloor M/B \rfloor$ blocchi.

Inoltre assumiamo che ogni blocco è leggibile in tempo unitario e ogni operazione intermedia (come operazioni aritmetiche) ha costo zero (difatti vogliamo modellare solo l'interazione con l'input/output).

Finalmente assumiamo la seguente relazione tra le quantità N, B, M :

$$N \gg M \gg B$$

Osservazione. Questo modello è anche utile per modellare l'interazione I/O con la cache e la RAM.

X

2. Case Studies

Facciamo un paio di esempi in cui analizziamo secondo il modello appena descritto.

#Esempio

Esempio 9 (Array Scan).

Consideriamo un file con N vettori interi, e vogliamo sommare tutti i valori contenuti.

Una cosa che possiamo fare è quello di leggere in blocchi di B byte e sommarli man mano e ottenere il risultato finale. Il numero delle operazioni I/O da fare è di grandezza $O(\lceil N/B \rceil)$

#Esempio

Esempio 10 (BST).

Supponiamo invece di voler salvare un albero binario con N elementi in memoria di massa.

Assumendo che l'albero è bilanciato, ci servono $\log_2 N$ operazioni per poter accedere ad una foglia. Tuttavia, se invece l'albero avesse un "*fan out*" k (non è più un albero binario), allora ci servirebbero $\log_k N$ operazioni, che sono considerabilmente di meno!

Infatti per $N = 10^9$ passiamo da $\log_2 10^9 \approx 30$ a $\log_{1000} 10^9 = 3$ operazioni!

Mappatura di Files in Memoria

X

Mappatura di files in memoria: idea preliminare, standard POSIX `mmap`. Signature della funzionalità, aspetti pratici ed esempi.

X

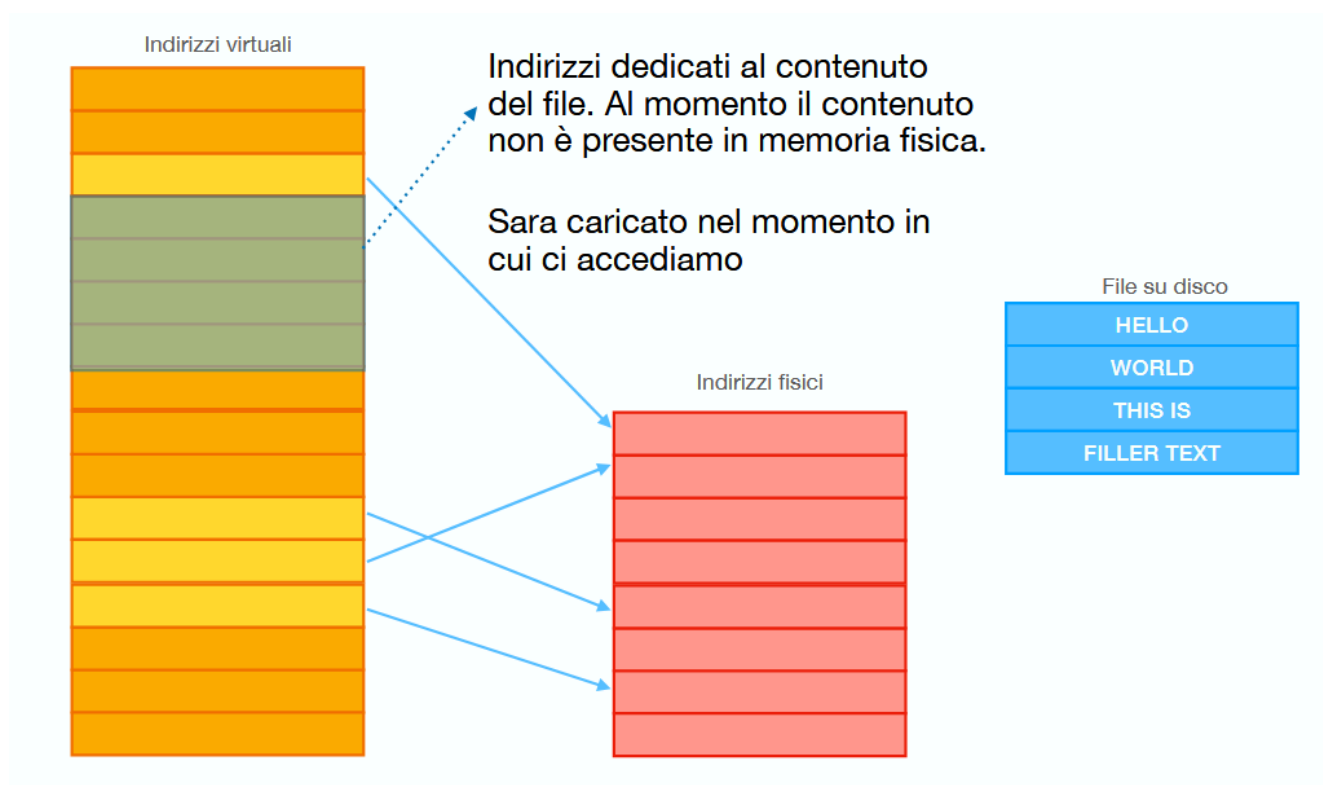
0. Voci correlate

- Richiami all'interazione coi files in C
- Modello del Processore con Memoria Esterna
- Layout della Memoria Virtuale
- Approcci Storici per l'Organizzazione della Memoria
- Richiami all'interazione coi files in C

1. Idea

Idea. Con le librerie standard si ha il sistema dei buffer e dei cursori, che potrebbe risultare "scomoda". Un'interfaccia alternativa più *trasparente* potrebbe consistere in trattare i file come se fossero dei vettori.

Ciò è reso possibile dalla *memoria virtuale*, ovvero in ogni programma che sta eseguendo abbiamo l'illusione di avere uno spazio molto grande di indirizzi *virtuali* di cui alcuni corrispondono ad indirizzi *fisici*.



Possiamo quindi fare finta che l'intero contenuto del file sia disponibile in memoria ad alcuni indirizzi virtuali e caricarne il contenuto *solo quanto serve*.

X

2. Funzionalità `mmap`

Lo standard POSIX prevede la funzionalità `mmap` per effettuare la mappatura di file in memoria. In particolare, ritorna un puntatore da cui posso leggere il file.

Per "caricare" questa funzionalità si include la libreria *non standard* (quindi è una cosa "in più") C: `<sys/mman.h>`. Vediamo la signature della funzionalità `mmap`.

Signature: `void* mmap(void* addr, size_t len, int prot, int flags, int fd, off_t offset);`

- Col campo `addr` si specifica l'indirizzo di memoria cui "mettere" il file; tipicamente si mette il puntatore nullo e ciò corrisponde a far decidere tutto al sistema operativo (il valore ritornato sarà quindi il puntatore deciso)
- Con `len` specifico la dimensione del mapping
- Con `prot` specifico delle costanti di protezione del file; sono costanti binarie e posso "metterli assieme" con un OR binario. Solitamente non si può mettere "execute" e "write" assieme per ovvi motivi di sicurezza
- Con `flags` specifico le varie flags, ci interesseranno le seguenti: `MAP_SHARED` e `MAP_PRIVATE`. Essa decide se le modifiche siano visibili dall'esterno o meno; nel primo caso sì, nel secondo no.
- Con `fd` specifico il file descriptor, quindi ovviamente servirà chiamare una funzione che apra un file (come `fopen` o `open`)
- Con `offset` specifico da dove iniziare il mapping

X

3. Aspetti pratici di `mmap`

Praticamente, per usare questa funzionalità si seguono i seguenti passaggi:

- Aprire il file con funzioni opportune e ottenere il suo *file descriptor*
- (Opzionale) Usare `stat()`; per ottenere informazioni opportune sul file; in particolare servirà il campo `st_size` per determinare la dimensione del mapping e se usare `ftruncate()`; o meno (N.B. Se creiamo un file "dal nulla" allora bisogna ridimensionarlo opportunamente con questa funzione)
- Chiamare `mmap()`;
- Effettuare le operazioni desiderate
- Usare `munmap()`; per rimuovere l'indirizzo di memoria mappata
- Finalmente "chiudere" il file

Questa funzionalità ha i suoi pro e contro:

- *Pro*: La funzionalità è molto comoda siccome ci affidiamo la gestione del file al sistema operativo, ed inoltre c'è la possibilità di farci programmazione multithread
- *Contro*: Dal fatto che affidiamo tutto al sistema operativo perdiamo efficienza, ed inoltre il processo di usare questa funzionalità è molto macchinosa; inoltre questa funzionalità funziona solo su sistemi POSIX, quindi non è portatile. Infine, i file non possono essere troppo grandi, infatti non deve superare lo spazio degli indirizzi virtuali (dipende dal sistema).

Osservazione. Facciamo un paio di note pratiche:

- I cambiamenti non vengono subito "*sincronizzati*", però si può forzarlo con la funzione `msync()`;
- Con `madvice` si può comunicare al sistema operativo con quale pattern accederemo al fail, come se fosse una sorta di prefetch
- Se ci salviamo dei puntatori non possiamo aspettarci che funzioneranno la prossima volta che useremo il programma; piuttosto si salvano gli *indirizzi relativi* rispetto all'inizio del mapping

X

4. Esempi di `mmap`

Per un esempio "*on the fly*" vedere il codice `Lecture 07/mmap.c`.

```
int main(int argc, char * argv[])
{
    char * data;
    struct stat sbuf;
    FILE * fd = fopen("mmap_demo", "r+");
    int fdnum = fileno(fd);

    stat("mmap_demo", &sbuf);

    data = mmap((void *)0, sbuf.st_size, PROT_READ, MAP_SHARED, fdnum, 0);

    for (int i = 0; i < 10; i++)
        printf("%c %c\n", data[i], getc(fd));

    munmap(data, sbuf.st_size);
    fclose(fd);

    return 0;
}
```

Un altro esempio in cui si usa `ftruncate()`; (`Lecture 07/mmap_vector.c`):


```
void write_vector(int len)
{
    FILE * fd = fopen("mmap_vector_demo", "w+");

    if (fd == NULL) {
        printf("Impossibile aprire il file\n");
        return;
    }

    int fdnum = fileno(fd);
    ftruncate(fdnum, len * sizeof(int));

    int * data = mmap((void *)0, len * sizeof(int), PROT_READ | PROT_WRITE,
MAP_SHARED, fdnum, 0);
    if (data == MAP_FAILED) {
        printf("Impossibile effettuare il mapping\n");
        return;
    }

    for (int i = 0; i < len; i++) {
        data[i] = i;
    }

    munmap(data, len * sizeof(int));
    fclose(fd);
}
```

SEZIONE C. PROGRAMMAZIONE VETTORIALE E PARALLELA

Tassonomia di Flynn

X

Motivazione per le istruzioni vettoriali. Nozioni preliminari: la tassonomia di Flynn (1972).

Tipologie di istruzioni: SISD, SIMD, MISD, MIMD e use cases. Tre tipi di SIMD:

Array/pipelined/associative processors.

X

0. Voci correlate

- Architettura di Von-Neumann

1. Tassonomia di Flynn (1972)

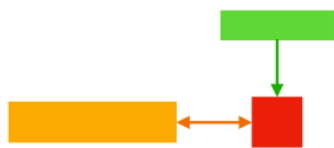
Motivazione. Consideriamo il seguente problema: siano $\underline{u}, \underline{v}$ due vettori e di calcolare la loro somma vettoriale $\underline{u} + \underline{v}$. Solitamente si fanno con un ciclo for sommando componente per componente. Possiamo fare di meglio? Sì! Essendo tutte operazioni "parallelizzabili" in quanto sono indipendenti e "uguali" tra loro, vogliamo cercare di parallelizzare questo procedimento.

Un metodo comune è quello di avere istruzioni che lavorano su k elementi alla volta.

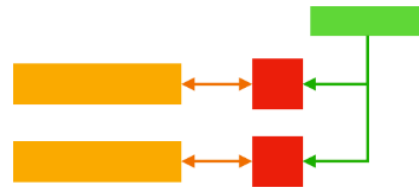
Tassonomia di Flynn (1972)

Formalmente, la tassonomia di Flynn classifica i flussi di istruzioni e dei dati in quattro categorie:

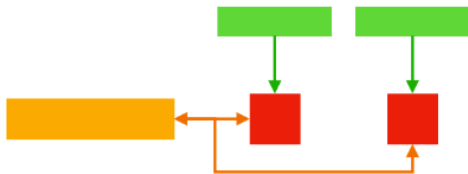
- *SISD*: Istruzioni "uno ad uno", quindi singolo flusso di dati e singolo flusso di istruzioni. Ovvero quello che abbiamo visto fino ad ora
- *SIMD*: Un singolo flusso di istruzioni ma più flussi di dati; è quello che vogliamo, infatti vedremo che ci sono tre sottocategorie di istruzioni SIMD
- *MISD*: Più flussi di dati ma singolo flusso di istruzioni. Oggi tendono ad avere un uso esotico e servono per verificare che più core "lavorino bene" nel senso che eseguendo le stesse istruzioni devono avere lo stesso risultato
- *MIMD*: Più flussi di dati e più flussi di istruzioni, servono per i *distributed systems*. Non li vedremo



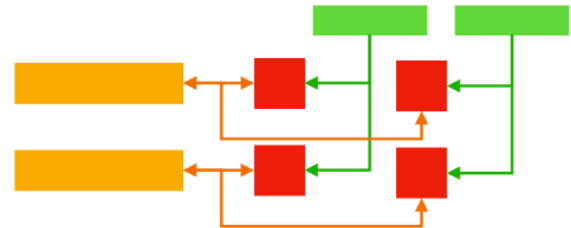
SISD
Single Instruction Stream
Single Data Stream



SIMD
Single Instruction Stream
Multiple Data Streams



MISD
Multiple Instruction Streams
Single Data Stream



MIMD
Multiple Instruction Streams
Multiple Data Streams

Tassonomia di Flynn, cont'ed

C'è un'ulteriore suddivisione di categorie SIMD:

- *Array processors*: Ogni unità riceve le stesse istruzioni ma opera su memorie separate
- *Pipelined processors*: Come prima ma su una memoria condivisa
- *Associative processors*: Ogni unità riceve le stesse istruzioni locali ma compiono una scelta se effettuarla o meno mediante array di bit. Sono anche noti come "*predicated instructions*"

Registri e Istruzioni Vettoriali

X

Breve descrizione qui

X

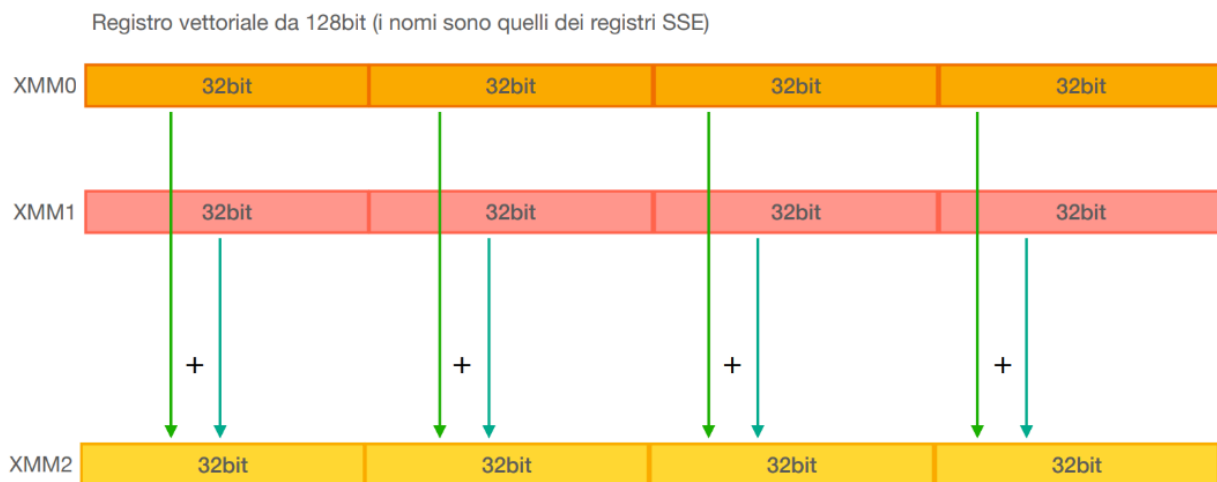
0. Voci correlate

- Tassonomia di Flynn

1. Estensioni Vettoriali

Note storiche. Dagli anni '90 diverse architetture hanno acquisito istruzioni e registri per lavorare su vettori di dati (come Cray 1 nel 1975 e CM-1 nel 1985)

Generalmente consistono in un insieme di registri *vettoriali* che devono essere implementati per più dati *dello stesso tipo* e un *insieme di istruzioni* (vettoriali) per lavorare su di essi.



Quindi si potrebbe usare direttamente *assembly* per lavorare con istruzioni vettoriali e ciò è comprensibilmente scomodo. Servono quindi diversi modi per usarli, e alcune opzioni sono:

1. Di non fare nulla
2. Di affidarsi al *compilatore* con le loro operazioni di *vettorizzazione automatica*. Molto utile per coprire i *casi comuni*; tuttavia non sono perfetti, soprattutto quando non siamo nei casi comuni
3. Usare le *intrinsics del compilatore*, ovvero delle loro estensioni che espongono le funzionalità "vettoriali" come funzioni. Tuttavia ciò rende il codice *non portabile* in quanto sono specifici su architetture
4. Usare estensioni vettoriali dei compilatori o librerie, in particolare per trattare le operazioni vettoriali mediante un tipo vettore generico.
5. Usare linguaggi con estensioni apposite per SIMD

La 2) si osserva facilmente usando *God bolt*, non vedremo la 5); al massimo accenneremo che essi implementano delle *keyword* come "uniform", "foreach" e altre variazioni per indicare che l'ordine è irrilevante. Invece ci soffermeremo sulle *intrinsic del compilatore* e le *estensioni vettoriali di GCC*.

Intrinsics del Compilatore

X

Intrinsics del compilatore: come si importano, come si compila. Alcuni esempi e alcune note pratiche. Estensioni vettoriali di GCC: motivazioni, idea ed esempi.

X

0. Voci correlate

- Tassonomia di Flynn
- Registri e Istruzioni Vettoriali

1. Intrinsics del Compilatore

N.B. Useremo le intrinsics di AVX, quindi per fare esempi è necessario usare un sistema operativo dotato di istruzioni AVX

"Intrinsics include C-style functions that provide access to other instructions without writing assembly code."

Ci sono tre step principali nella compilazione di un programma con le *intrinsics*

1. Per importare le intrinsics bisogna prima necessariamente importare le librerie necessarie (nel caso delle intrinsics AVX è `<immintrin.h>`).
2. Dopodiché per usarle si usano *datatype* e *funzioni* apposite, come ad esempio nel caso di AVX il tipo `__m256` è "8 float di 32 bit" e `__m256 _mm256_add_ps(__m256 x, __m256 y)` è "sommare due 8 vettori floating point da 32 bit".
3. Infine si compila il programma includendo `-max` o un'opzione di march che lo implica.

Osservazione. Bisogna compilare anche col flag `-O0` per evitare autovettorizzazioni (se lo scopo principale del programma è quello di usare principalmente le intrinsics)!

Per un esempio vedere il codice `/Lecture 09/avx.c`.

```

int main(int argc, char * argv[])
{
    const int n = 1<<20;
    float * a = make_array(n);
    float * b = make_array(n);
    float * c = make_array(n);
    clock_t start = clock();

    for (int i = 0; i < n; i += 8) {
        __m256 x = _mm256_loadu_ps(&a[i]);
        __m256 y = _mm256_loadu_ps(&b[i]);
        __m256 z = _mm256_add_ps(x, y);
        _mm256_storeu_ps(&c[i], z);
    }

    clock_t end = clock();
    float ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);

    printf("Somma con istruzioni vettoriali: %f ms\n", ms);

    start = clock();

    for (int i = 0; i < n; i++)
        c[i] = a[i] + b[i];

    end = clock();

    ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);

    printf("Somma senza istruzioni vettoriali: %f ms\n", ms);

    return 0;
}

```

Per una compilazione estensiva sulle intrinsics AVX vedere il seguente sito:

<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#>

2. Estensione Vettoriale GCC

Un altro approccio è quello di usare le *estensioni vettoriali generiche di GCC*, rendendo in questo modo il codice portatile su più architetture.

Si possono definire i *datatype* con sintassi simili, come ad esempio `typedef int v4si`
`__attribute__((vector_size(4*sizeof(int))))`; per definire un vettore di interi a 4 byte con segno. Dopodiché sarà il compito del compilatore capire *quali* istruzioni vettoriali usare.

Per un esempio vedere il codice `/Lecture 09/gcc_extensions.c`, che fa somme su vettori

```
typedef int v8si __attribute__((vector_size (8 * sizeof(int))));

int sum_vector_simd(int * vec, int len)
{
    v8si sumv = {0, 0, 0, 0, 0, 0, 0, 0};
    int i;

    for (i = 0; i+7 < len; i+= 8) {
        v8si x = *(v8si *) &vec[i];
        sumv += x;
    }

    int sum = sumv[0] + sumv[1] + sumv[2] + sumv[3] + sumv[4] + sumv[5] + sumv[6] +
sumv[7];

    for (; i < len; i++)
        sum += vec[i];

    return sum;
}
```


Programmazione Multithreading con OpenMP

X

Motivazioni e note storiche sulla programmazione multithread. Come compilare un programma con OpenMP. Alcune direttive OpenMP: `parallel`, `single`, `critical`, `for` e `nowait`. Funzioni utili per distinguere thread. Scheduling di thread. Parallelizzare cicli innestati. Task-based parallelism.

X

0. Voci correlate

- Concorrenza e Parallelismo
- Concetto di Thread
- Registri e Istruzioni Vettoriali
- <https://ppc.cs.aalto.fi/ch3/>

1. Programmazione Multithread

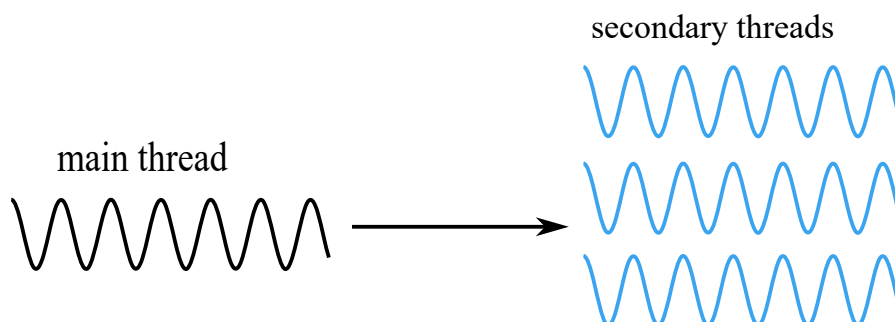
Fino ad ora avevamo parallelizzato alcune operazioni con *registri vettoriali*. Tuttavia si potrebbe fare molto di meglio, infatti solitamente le macchine moderne hanno *più di un core*! Sarebbe quindi bene sfruttarli con la *programmazione multithread*.

Note storiche. La programmazione multithread era nata circa 30 anni fa in *ambito supercomputer* con sistemi multiprocessori con memoria condivisa; servivano principalmente per *simulare sistemi complessi*.

Faremo la programmazione multithread con *OpenMP*.

OpenMP è un'API per il supporto della programmazione parallela in sistemi multi-processore e memoria condivisa che fa uso di thread. La struttura consiste in un *thread principale* che fa partire più *thread secondary* e il sistema distribuisce il lavoro tra più thread. Per distinguere i thread tra loro, si assegnano valori interi a loro.

Per importare OpenMP si importa la libreria `<omp.h>` e bisogna compilare il programma col flag `-fopenmp`.



Vedremo una serie di costrutti utili per la parallelizzazione.

2. Funzioni per Thread

Prima di vedere *come* creare blocchi di codici da eseguire in parallelo, vediamo un paio di funzioni utili della libreria.

1. `omp_get_max_threads()`: Per ottenere il numero di thread che verranno usati nelle sezioni parallele
2. `omp_get_num_threads()`: Per ottenere il numero di thread che OpenMP sta usando nella sezione parallel corrente
3. `omp_get_thread_num()`: Per ottenere l'identificativo del thread corrente.

3. Direttive Pragma OpenMP Principali

La maggior parte delle funzionalità OpenMP sono legate alla direttiva `#PRAGMA`, che serve per fornire informazioni aggiuntive al compilatore (notiamo che quindi il supporto di OpenMP è legato al compilatore!)

Vediamo una serie di direttive utili di OpenMP.

3.1. Sezioni OpenMP

1. `#pragma omp parallel [num_threads(n)]`: Serve per indicare che il seguente blocco di codice è parallelizzabile, quindi bisogna copiare su vari thread. A quel punto le variabili saranno *locali rispetto ai thread*. Opzionalmente si può specificare *n* il numero di thread da creare

```
a();
#pragma omp parallel
{
    b();
    c();
}
d();
```

Thread 0:



Thread 1:



Thread 2:



Thread 3:



Tempo →

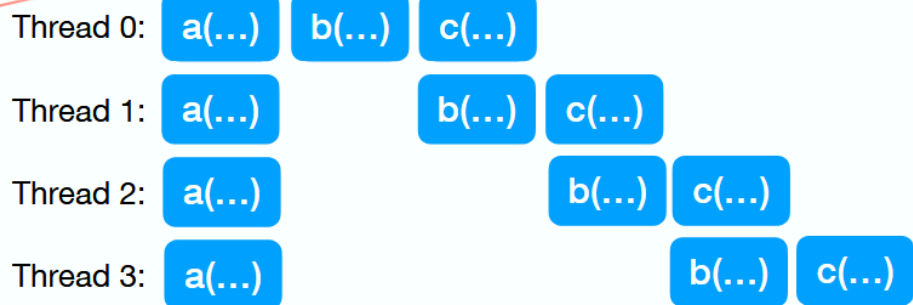
2. `#pragma omp single`: In casi particolari è necessario specificare che una funzionalità eseguita in un blocco parallelo è da eseguire una sola volta

Osservazione. Modificare variabili definire *fuori* dal blocco `PARALLEL` porta alle "race conditions", ovvero problemi legati al parallelismo. Per evitare di rischiare di perdere informazioni, si ha una direttiva per definire le "sezioni critiche" che vengono protette da meccanismi di lock.

3. **#pragma omp critical**: Per definire le *sezioni critiche*, quindi sezioni da eseguire per tutti i thread ma "solo una alla volta".

```
#pragma omp parallel
{
    a()
    #pragma omp critical
    {
        b();
    }
    c();
}
```

Eseguita da un thread alla volta
(ma tutti i thread la eseguono)!



Tempo

3.2. Cicli For

4. **#pragma omp parallel for** f: Serve per parallelizzare i cicli for, in particolare quelli in cui ogni iterazione agisce su dati diversi (come indici diversi di array).

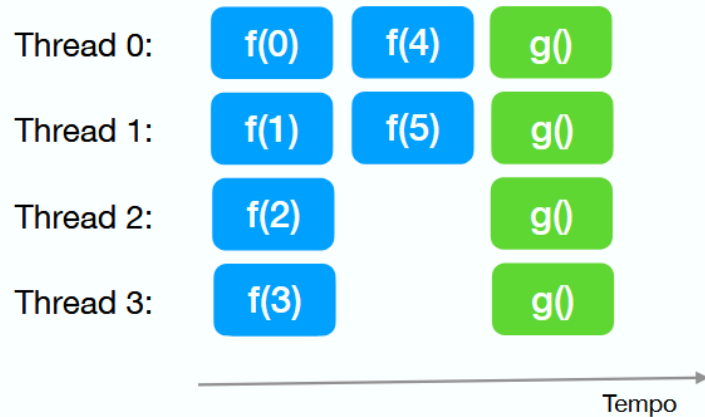
```
#pragma omp parallel for
for (int i = 0; i < 16; i++) {
    f(i);
}
```

Dividi le iterazioni del ciclo for
tra i diversi thread



Osservazione. Quest'ultima direttiva è la combinazione di due pragma, ossia **PARALLEL** e **FOR**. Il secondo serve solo per dividere l'esecuzione delle diverse iterazioni del ciclo for tra diversi tread. In alcuni casi, si può dividere le due direttive

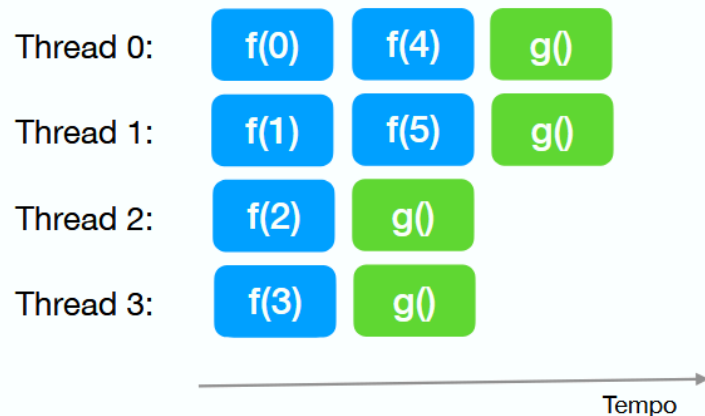
```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < 6; i++) {
        f(i);
    }
    g();
}
```



Osservazione. Con il **PARALLEL FOR** i thread attendono che tutti gli altri thread finiscano di eseguire le loro istruzioni (quindi che l'iterazione for sia completa). Tuttavia in certi casi non è necessaria questa attesa, da cui la seguente direttiva:

5. **#pragma omp nowait**: Per impedire attese come osservato sopra.

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (int i = 0; i < 6; i++) {
        f(i);
    }
    g();
}
```



Osservazione. Un caso molto comune dell'utilizzo dei cicli for è quello di effettuare dei calcoli su un insieme di elementi e calcolare una sua specie di somma cumulativa. Generalmente, questa è un'operazione di riduzione. Per farlo bisogna stare particolarmente attenti alle *race condition* definendo alcune sezioni critiche, eccetera...

La seguente direttiva automatizza questa operazione.

6. `#pragma omp reduction(op: var)`: Si legge "riduzione tramite `op` su `var`", effettua l'operazione di riduzione. Le operazioni sono binarie, solitamente `+`, `-`, `*`, `&`, `|`, `...`. Tuttavia ci sono alcune limitazioni sulla variabile target `var`; in particolare, solo le seguenti forme sono sopportate:

- `var = var op expr` (l'espressione non deve dipendere dalla variabile)
- `var = expr op var`
- `var op = expr`
- `var ++`, `var--`, `++var`, `--var`

X

4. Scheduling

Osservazione. Di default con le `PARALLEL FOR` le iterazioni vengono "equamente" distribuite tra thread, ossia la prima iterazione al primo thread, la seconda al secondo ed eccetera...

In realtà questa è una forma di *scheduling* statico con dimensione 1. Si potrebbe cambiare questo sistema di assegnazione delle iterazioni con la *schedule*.

In primo luogo si può cambiare la *dimensione* arbitrariamente, quindi al posto di dare un'iterazione per thread se ne danno k . Ciò si fa specificando nella direttiva pragma `schedule(static, k)`

```
#pragma omp parallel
{
    #pragma omp for schedule(static,2)
    for (int i = 0; i < 12; i++) {
        f(i);
    }
    g();
}
```

Thread 0:

f(0), f(1)

f(8), f(9)

Thread 1:

f(2), f(3)

f(10), f(11)

Thread 2:

f(4), f(5)

Thread 3:

f(6), f(7)

Tempo →

Alternativamente c'è anche lo *schedule dinamico*, ossia invece di assegnare le iterazioni staticamente si crea una coda di iterazioni da assegnare che viene "man mano" liberata dai thread.

Tuttavia quest'ultima forma di schedule è *costosa*, quindi viene fatta solo quando c'è disparità nelle iterazioni. Ancora più generalmente, bisogna trovare un buon *trade off*. Si trova il valore giusto misurando, siccome questo è un problema dipendente dai dati.

```
#pragma omp parallel
```

```
{
```

```
    #pragma omp for schedule(dynamic,1)
```

```
    for (int i = 0; i < 12; i++) {
```

```
        f(i);
```

```
    }
```

```
    g();
```

```
}
```

Thread 0:

f(0)

f(5)

f(10)

Thread 1:

f(1)

f(9)

Thread 2:

f(2)

f(4)

f(8)

f(11)

Thread 3:

f(3)

f(6)

f(7)

Tempo

X

5. Cicli Innestati

Q. Avevamo parallelizzato i cicli for singoli con la direttiva `#pragma omp parallel`. Come gestiamo invece i cicli innestati?

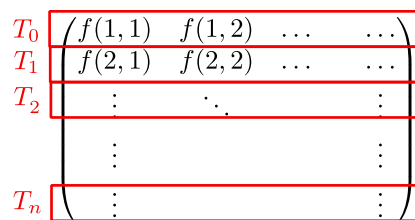
Questo è un caso particolare che merita particolare attenzione, siccome ci sono più approcci per gestire questo caso.

#Esempio

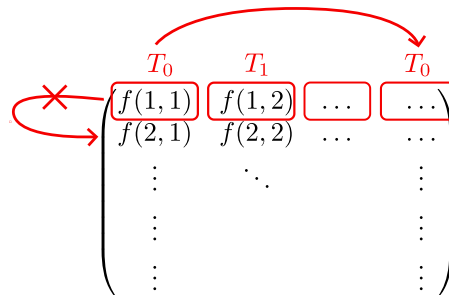
Esempio. Come esempio motivante, ricordiamo che l'algoritmo "naïve" della moltiplicazioni di matrici richiede l'innestamento di tre cicli for. Infatti, ricordiamo che

$$(ab)_{ij} = \sum_{k=0, \dots, n} a_{ik} b_{kj}$$

Approccio 1. Se le iterazioni sono equilibrate, allora solitamente va bene mettere la sezione parallela *solo* sul ciclo esterno. In un certo senso, stiamo assegnando ad ogni thread una "riga" della "matrice delle operazioni da effettuare". Tuttavia, se sono squilibrate allora non va bene siccome i carichi dei lavori di ogni thread non possono essere condivisi: ciò crea quindi un collo di bottiglia sul thread con la "riga più pesante".



Approccio 2. Parallelizzando invece il *ciclo interno*, abbiamo che ogni "riga" viene condivisa ma tutti i thread aspettano che questa riga venga completata. Quindi abbiamo ancora un altro collo di bottiglia...



Nessuno dei due approcci è ottimale, vogliamo qualcosa che ci faccia dividere *tutte* le iterazioni come se fosse un unico ciclo for.

Per capire bene il nuovo approccio, osserviamo il seguente esempio.

#Esempio

Esempio. Supponiamo di avere un array bidimensionale $V[n-1, m-1]$ e di voler calcolare la sua somma. In particolare, per accedere alla sua (i, j) -esima posizione accediamo a $V[im + j]$. Poniamo quindi $k := im + j$, tuttavia notiamo che non è altro che la divisione euclidea in $\mathbb{Z}$ di k per m , quindi si ha che $i = k/m$ e $j = k \bmod m$. In questo modo, possiamo iterare da $k = 0, \dots, mn - 1$ in un singolo ciclo for e comunque accedere a tutti gli elementi di V .

La direttiva `#pramga omp collapse(n)` fa proprio questo, dove n è il numero di cicli più esterni di cui collassare in un unico ciclo.

Osservazione. Nell'usare questa direttiva, bisogna stare molto attenti a quali strati parallelizziamo: ad esempio, se nello strato più interno richiede un certo ordine da rispettare, allora sarebbe meno opportuno parallelizzare quest'ultima parte.

X

6. Parallelismo "Task-Based"

Idea. Abbiamo un insieme di "compiti da fare" di cui assegnare ad un "pool" di thread. L'idea del parallelismo "task-based" è quello di avere un unico thread che distribuisce i compiti ad altri thread, che poi li eseguiranno.

Per definire le *task* si usa la direttiva `#pragma omp task`.

Osservazione. Prima di definire le task, è definire necessario il blocco *parallelo* e il blocco *singolo* (!), siccome un thread distribuisce i compiti e gli altri li eseguono.

```

#pragma omp parallel
#pragma omp single
{
    f(2);
#pragma omp task
    g(3);
#pragma omp task
    f(12);
    g(24);
    f(3);
#pragma omp task
    g(5);
    g(8);
}

```

Thread 0:

f(2)

g(24)

f(3)

g(8)

Thread 1:

g(3)

g(5)

Thread 2:

f(12)

Thread 3:

Tempo →